

Esercizi di Architetture dei Sistemi Digitali

Raffaele Carillo - Mat. M63/001321

12 ottobre 2023

Indice

1	Multiplexer	1
1.1	Esercizio 1.1	1
1.1.1	Schematici	1
1.1.2	Codice	2
1.1.3	Simulazione	3
1.2	Esercizio 1.2	3
1.2.1	Schematici	4
1.2.2	Codice	4
1.2.3	Simulazione	5
1.3	Esercizio 1.3	5
1.3.1	Sintesi su board FPGA	6
2	Encoder BCD	7
2.1	Esercizio 2.1	7
2.1.1	Codice	7
2.1.2	Schematici	8
2.1.3	Simulazione	8
2.2	Esercizio 2.2	8
2.2.1	Sintesi su board FPGA	9
2.3	Esercizio 2.3	9
2.3.1	Codice	9
2.3.2	Schematici	11
2.3.3	Sintesi su board FPGA	11
3	Riconoscitore di sequenza	13
3.1	Schematici	13
3.2	Codice	14
3.3	Simulazione	16
3.4	Sintesi su board FPGA	16
4	Shift Register	17
4.1	Approccio comportamentale	17
4.1.1	Codice	17
4.1.2	Schematici	18
4.1.3	Simulazione	19
4.2	Approccio strutturale	20

4.2.1	Schematici	20
4.2.2	Simulazione	21
5	Cronometro	22
5.1	Esercizio 5.1	22
5.1.1	Unità operativa - Orologio	22
5.1.1.1	Contatore	22
5.1.1.2	Schematici	23
5.1.1.3	Simulazione	24
5.2	Esercizi 5.2 e 5.3	24
5.2.1	Schematici	25
5.2.2	Convertitore Binario-BCD	25
5.2.3	RAM	26
5.2.4	Unità di Controllo Display	26
5.2.4.1	Codice	27
5.2.4.2	Simulazione	28
5.2.5	Unità di Controllo Orologio	28
5.2.5.1	Simulazione	29
5.2.6	Sintesi su board FPGA	30
6	Sistema di Testing	31
6.1	Esercizio 6	31
6.1.1	Schematici	31
6.1.2	Codice	32
6.1.3	Simulazione	32
7	Handshake	33
7.1	Schematici	33
7.2	Sistema A	35
7.2.1	Codice	35
7.2.2	Simulazione	36
7.3	Sistema B	36
7.3.1	Codice	36
7.3.2	Simulazione	37
7.4	Interfaccia	37
7.4.1	Codice	38
7.4.2	Simulazione	40
7.5	Simulazione	40
8	Processore	41
8.1	Analisi di ISTORE	41
8.2	Analisi di BIPUSH	45
8.3	Modifica di IADD	47
8.4	Istruzioni di I/O	49
8.5	Sintesi su board FPGA	50
8.5.1	Schematici	50

8.5.2	Codice VHDL	51
8.5.3	Codice IJVM	52
9	Seriale	53
9.1	Esercizio 9.1	53
9.1.1	Schematici	53
9.1.2	Codice	53
9.1.3	Simulazione	55
9.1.4	Sintesi su board FPGA	55
9.2	Esercizio 9.2b	55
9.2.1	Schematici	56
9.2.2	Sintesi su board FPGA	56
10	Divisore	58
10.1	Schematici	58
10.2	Codice	58
10.3	Simulazione	60
10.4	Sintesi su board FPGA	61
11	Switch Multistadio	62
11.1	Schematici	62
11.2	Codice VHDL	63
11.3	Simulazione	63
12	Rete Neurale (Esercizio Libero)	64
12.1	Perceptron	64
12.1.1	Schematici	65
12.1.2	Codice	65
12.1.3	Simulazione	67
12.2	Rete neurale	68
12.2.1	Schematici	68
12.2.2	Codice	69
12.2.3	Simulazione	70

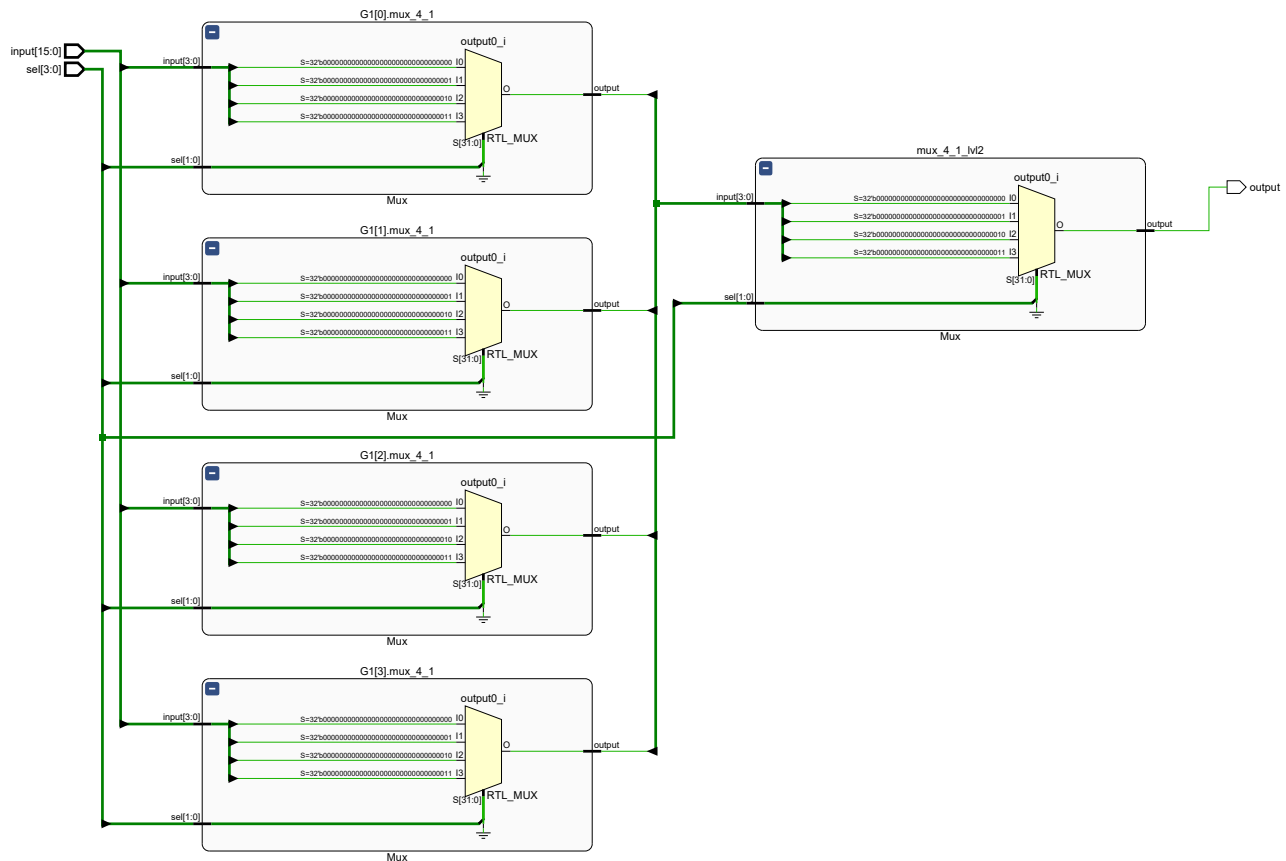
Capitolo 1

Multiplexer

1.1 Esercizio 1.1

Progettare, implementare in VHDL e testare mediante simulazione un multiplexer indirizzabile 16:1, utilizzando un approccio di progettazione per composizione a partire da multiplexer 4:1.

1.1.1 Schematici



Un multiplexer 16:1 può essere realizzato componendo cinque multiplexer 4:1. Al primo livello abbiamo quattro multiplexer, ognuno dei quali riceve quattro dei segnali di ingresso, a tutti i

multiplexer di questo livello vengono connessi i primi due bit di selezione. L'uscita del primo livello viene connessa a un multiplexer che sceglierà l'uscita del primo livello usando gli altri due bit di selezione e la passerà in output.

1.1.2 Codice

```

1  entity Mux_16_1 is
2      Port ( output : out STD_LOGIC;
3            sel : in STD_LOGIC_VECTOR (3 downto 0);
4            input : in STD_LOGIC_VECTOR(15 downto 0));
5  end Mux_16_1;
6
7  architecture Structural of Mux_16_1 is
8  component Mux
9  generic(
10     N: integer := 1;
11     M: integer := 2
12 );
13     -- N: numero di ingressi
14     -- M: bit per ingresso e di output
15 port(
16     input: in std_logic_vector(N*M-1 downto 0);
17     sel: in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
18     output: out std_logic_vector(M-1 downto 0)
19 );
20 end component;
21 signal level_1_out: std_logic_vector(3 downto 0) := "0000";
22 begin
23
24
25 G1: for i in 0 to 3 generate
26 mux_4_1: Mux generic map (N => 4, M => 1)
27 port map(
28     input => input((i+1)*4-1 downto i*4),
29     output(0) => level_1_out(i),
30     sel => sel(1 downto 0)
31 );
32 end generate G1;
33
34 mux_4_1_lvl2: Mux generic map (N => 4, M => 1)
35 port map(
36     input => level_1_out,
37     output(0) => output,
38     sel => sel(3 downto 2)
39 );
40
41 end Structural;
```

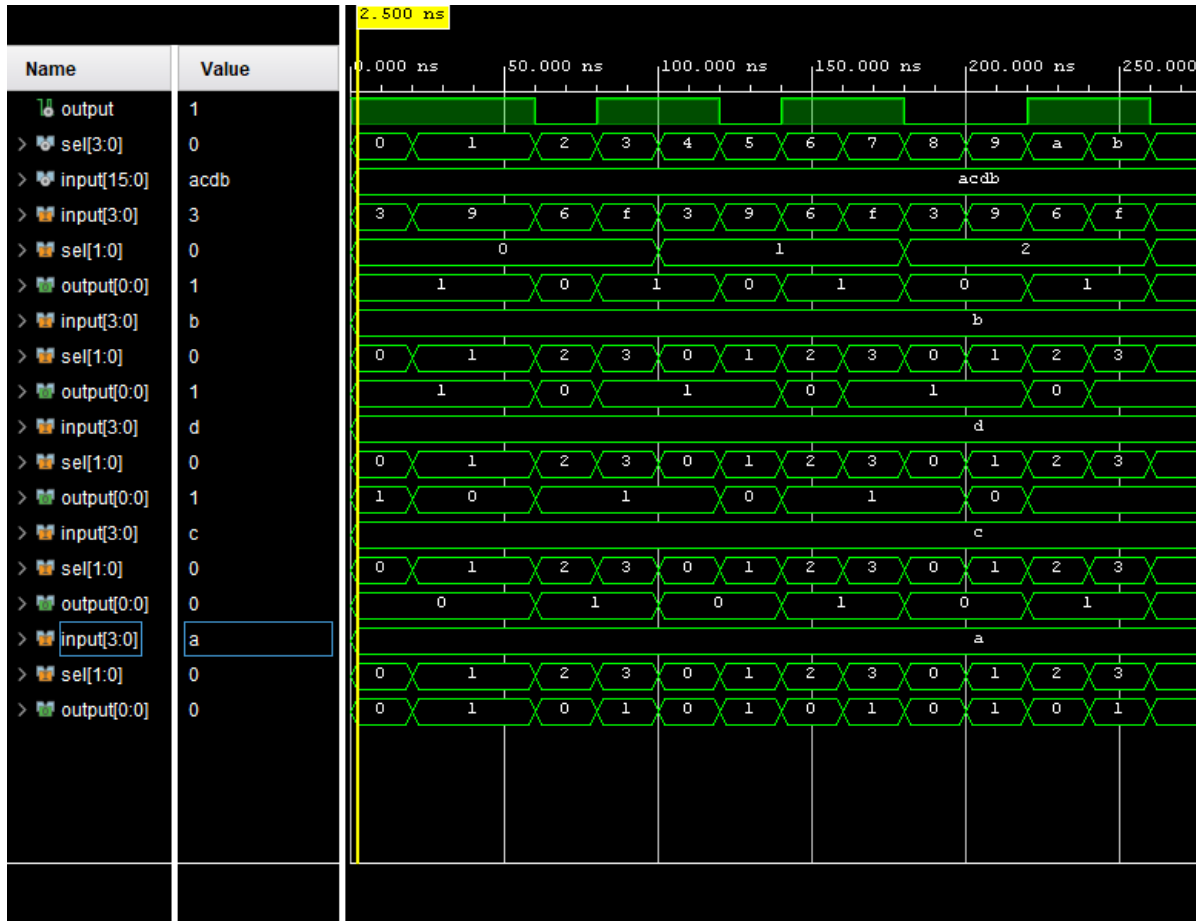
I mux al primo livello sono stati creati per compattezza del codice usando l'istruzione generate e assegnando a ognuno di essi 4 dei 16 segnali di ingresso.

Ogni mux è realizzato mediante un generic che verrà riutilizzato successivamente per gli altri esercizi.

```

1  entity Mux is
2  generic(
3      N: integer := 1;
4      M: integer := 2
5  );
6      -- N: numero di ingressi
7      -- M: bit per ingresso e di output
8  port(
9      input: in std_logic_vector(N*M-1 downto 0);
10     sel: in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
11     output: out std_logic_vector(M-1 downto 0)
12 );
13 end Mux;
14
15 architecture rtl of Mux is
16
17 begin
18
19 output <= input(M*(to_integer(unsigned(sel))+1)-1 downto M*(to_integer(unsigned(sel))));
20
21 end rtl;
```

1.1.3 Simulazione

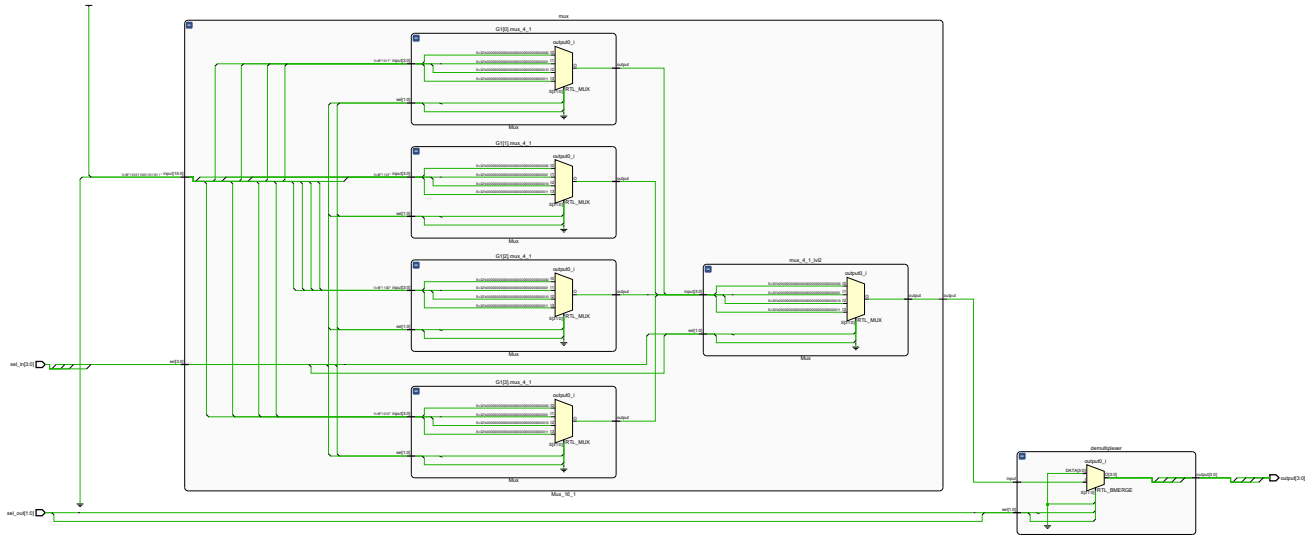


In input viene passato il valore esadecimale 0xACBD per il testbench vengono selezionati uno per volta i primi 12 input verificando quindi il funzionamento dei due livelli di selezione.

1.2 Esercizio 1.2

Utilizzando il componente sviluppato al punto precedente, progettare, implementare in VHDL e testare mediante simulazione una rete di interconnessione a 16 sorgenti e 4 destinazioni.

1.2.1 Schematici



A valle dell'uscita del multiplexer 16:1 viene aggiunto un demultiplexer 4:1 collegato a due opportuni segnali di selezione. Un `std_logic_vector` di dimensione 4 viene usato come uscita.

1.2.2 Codice

Lo switch è stato creato secondo un approccio strutturale combinando il multiplexer 16:1 creato precedentemente e un demultiplexer 4:1.

```

1  entity Switch is
2  Port (
3      input : in std_logic_vector(15 downto 0) := X"ACDB";
4      output: out std_logic_vector(3 downto 0);
5      sel_in: in std_logic_vector(3 downto 0);
6      sel_out: in std_logic_vector(1 downto 0)
7  );
8  end Switch;
9
10 architecture Structural of Switch is
11 component Mux_16_1
12     Port ( output : out STD_LOGIC;
13           sel : in STD_LOGIC_VECTOR (3 downto 0);
14           input : in STD_LOGIC_VECTOR(15 downto 0));
15 end component;
16 component DeMux
17 generic(
18     N: integer := 1;
19     M: integer := 2
20 );
21     -- N: numero di uscite
22     -- M: bit per ingresso e di output
23 port(
24     input: in std_logic_vector(M-1 downto 0);
25     sel: in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
26     output: out std_logic_vector(N*M-1 downto 0)
27 );
28 end component;
29 signal mux2demux: std_logic := '0';
30 begin
31
32 mux: Mux_16_1 port map(
33     input => input,
34     output => mux2demux,
35     sel => sel_in
36 );
37
38 demultiplexer: DeMux generic map(N=>4, M=> 1) port map(
39     input(0) => mux2demux,
40     output => output,
41     sel => sel_out
42 );
43
44 end Structural;

```

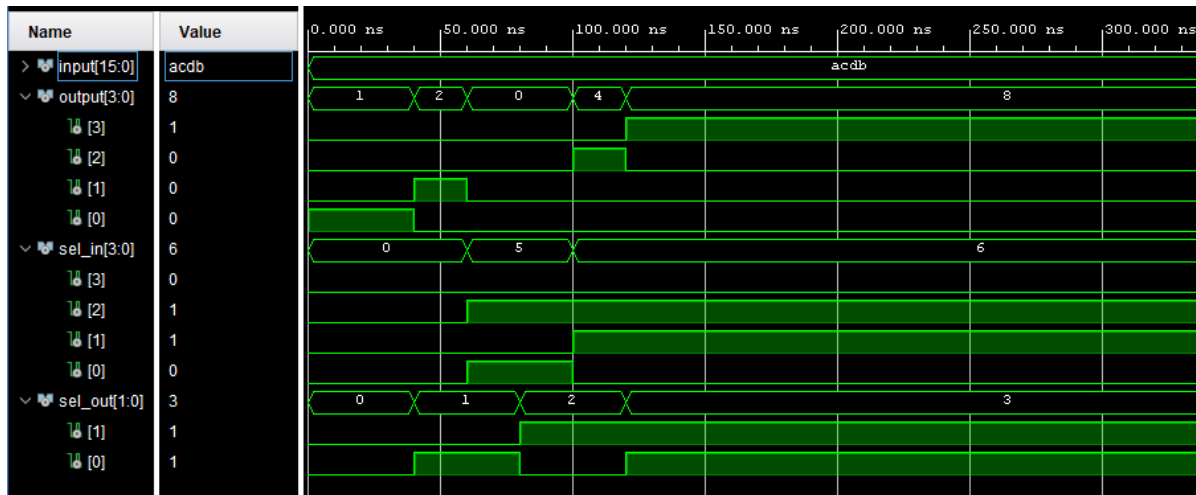
Anche il demultiplexer è stato creato usando sfruttando i generics in modo tale da riutilizzarlo per progetti futuri.

```

1  entity DeMux is
2  generic(
3      N: integer := 1;
4      M: integer := 2
5  );
6      -- N: numero di uscite
7      -- M: bit per ingresso e di output
8  port(
9      input: in std_logic_vector(M-1 downto 0);
10     sel: in std_logic_vector(integer(ceil(log2(real(N))))-1 downto 0);
11     output: out std_logic_vector(N*M-1 downto 0)
12 );
13 end DeMux;
14
15 architecture rtl of DeMux is
16
17 begin
18
19 process (sel, input)
20     variable tmp: std_logic_vector(N*M-1 downto 0);
21 begin
22     tmp := (others => '0');
23     tmp(M*(to_integer(unsigned(sel))+1)-1 downto M*(to_integer(unsigned(sel)))) := input;
24     output <= tmp;
25
26 end process;
27 end rtl;

```

1.2.3 Simulazione



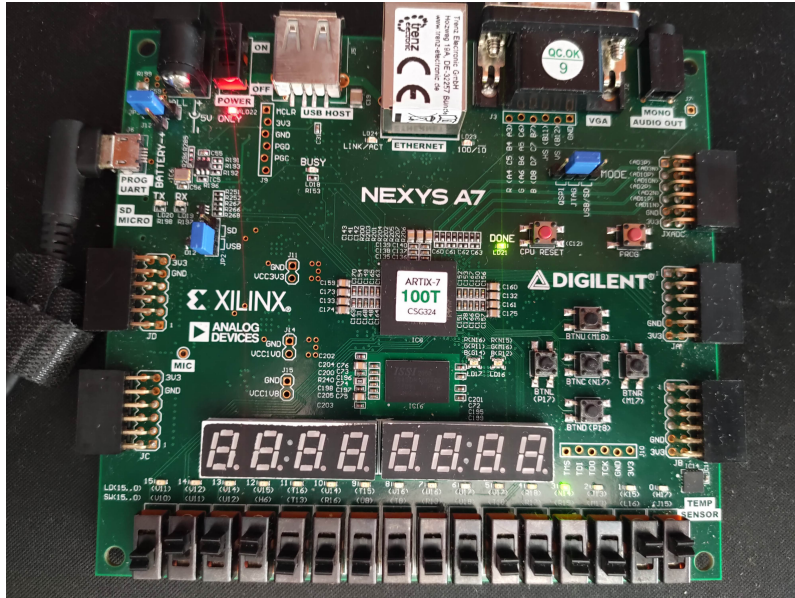
Anche per lo switch è stata fatta una simulazione, in particolare il primo ingresso viene inviato prima alla prima e poi alla seconda uscita, il sesto ingresso viene inviato alla terza uscita e infine il settimo ingresso viene passato alla quarta uscita.

1.3 Esercizio 1.3

Sintetizzare ed implementare su board il progetto della rete di interconnessione sviluppato al punto 1.2, utilizzando gli switch per fornire gli input di selezione e i led per visualizzare i 4 bit di uscita. Per quanto riguarda i 16 bit dato in input, essi possono essere precaricati nel sistema oppure immessi anch'essi mediante switch, sviluppando in questo secondo caso un'apposita rete di controllo per l'acquisizione.

1.3.1 Sintesi su board FPGA

Viene precaricato il solito ingresso (0xacdb) e mediante i constraint vengono associati i primi due switch alla selezione del demultiplexer e gli ultimi 4 switch alla selezione in ingresso del demultiplexer. Allo stesso modo l'output dello switch viene collegato ai LED della board.



Capitolo 2

Encoder BCD

2.1 Esercizio 2.1

Progettare, implementare in VHDL e testare mediante simulazione una rete che, data in ingresso una stringa binaria X di 10 bit $X_9 X_8 X_7 X_6 X_5 X_4 X_3 X_2 X_1 X_0$ che corrisponde alla rappresentazione decodificata di una cifra decimale (cioè, una rappresentazione in cui ogni stringa contiene un solo bit alto), fornisce in uscita la rappresentazione Y della cifra mediante codifica Binary-Coded Decimal (BCD).

Input: 0000000001 Output: 0000 (cifra 0)

Input: 0000000010 Output: 0001 (cifra 1)

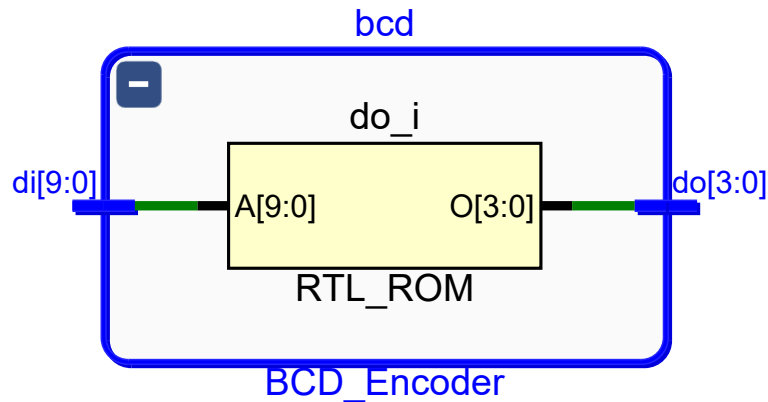
Input: 0000000100 Output: 0010 (cifra 2)

2.1.1 Codice

```
1  entity BCD_Encoder is
2      Port (
3          di: in std_logic_vector(9 downto 0);
4          do: out std_logic_vector(3 downto 0)
5      );
6  end BCD_Encoder;
7
8  architecture rtl of BCD_Encoder is
9
10     begin
11
12     with di select do <=
13         "0000" when "0000000001",
14         "0001" when "0000000010",
15         "0010" when "0000000100",
16         "0011" when "0000001000",
17         "0100" when "0000010000",
18         "0101" when "0000100000",
19         "0110" when "0001000000",
20         "0111" when "0010000000",
21         "1000" when "0100000000",
22         "1001" when "1000000000",
23         "0000" when others;
24
25     end rtl;
```

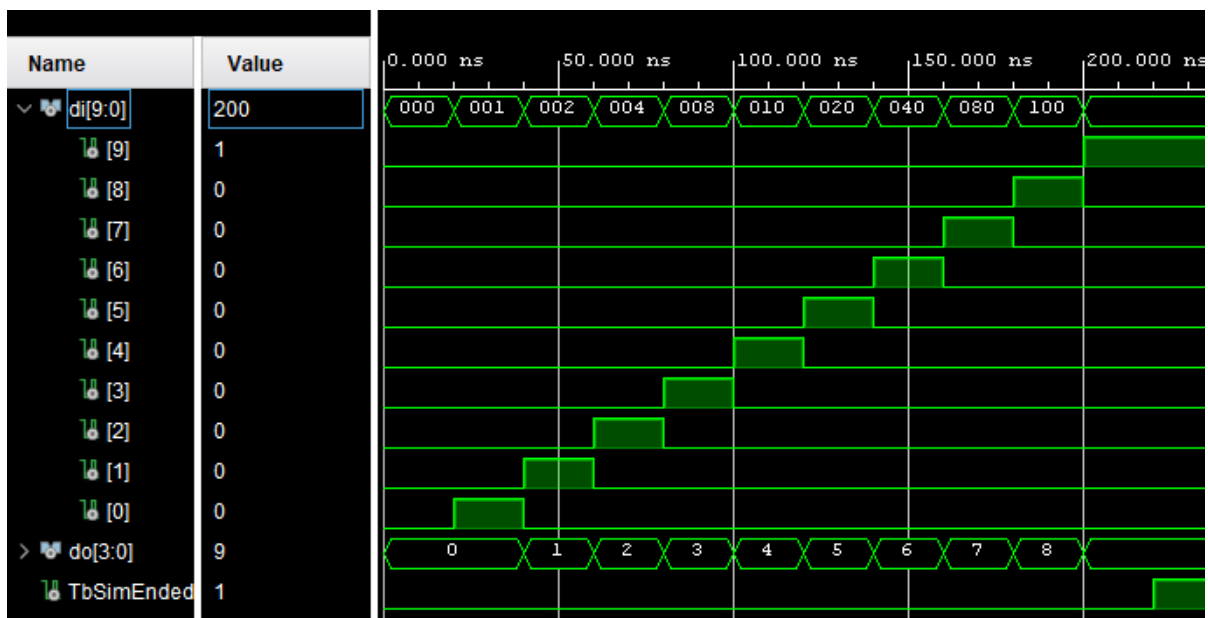
Viene usato il costrutto `with...select` per associare ad ognuno degli ingressi la rappresentazione binaria corrispondente della posizione alla quale il bit in ingresso è alto.

2.1.2 Schematici



Il BCD Encoder viene implementato come una semplice memoria ROM che associa ai dati di ingresso dei valori in uscita.

2.1.3 Simulazione



Alziamo uno per volta i bit in ingresso al BCD Encoder e come ci si aspetterebbe in uscita vi è la rappresentazione binaria del numero corrispondente.

2.2 Esercizio 2.2

Sintetizzare ed implementare su board il progetto dell'encoder BCD utilizzando gli switch per fornire la stringa X in ingresso, e i led per visualizzare Y. Nel caso in cui si utilizzi una board dotata di

soli 8 switch, è possibile sviluppare il progetto considerando X di soli 8 bit (la macchina sarà allora in grado di fornire in uscita la rappresentazione BCD delle cifre decimali da 0 a 7).

2.2.1 Sintesi su board FPGA

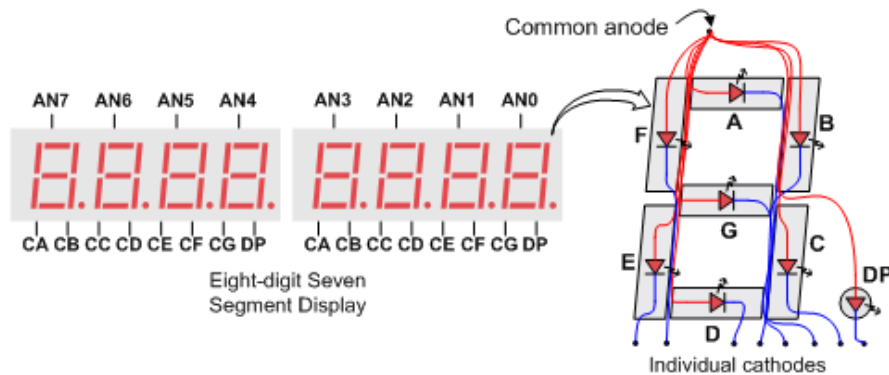
Anche in questo caso è sufficiente associare all'ingresso del BCD Encoder gli switch e all'uscita i LED della board.

2.3 Esercizio 2.3

Utilizzare un display a 7 segmenti per visualizzare la cifra decimale codificata da Y (pilotare opportunamente i catodi del display per visualizzare la cifra).

2.3.1 Codice

Trattandosi del primo esercizio in cui viene sfruttato il display verrà analizzato nel dettaglio il pilotaggio degli anodi e catodi per visualizzare le cifre del BCD.



Ognuno degli 8 display a 7 segmenti della board è possiede un anodo comune e un catodo per ognuno dei segmenti. Quando vogliamo visualizzare un carattere verranno attivati gli anodi (col livello logico alto) e i catodi (col livello logico basso). Vediamo come viene effettuato il pilotaggio degli anodi.

```

1  entity anodes_manager is
2      Port ( counter : in  STD_LOGIC_VECTOR (2 downto 0);
3            enable_digit : in  STD_LOGIC_VECTOR (7 downto 0);
4            anodes : out  STD_LOGIC_VECTOR (7 downto 0)
5          );
6  end anodes_manager;
7
8  architecture Behavioral of anodes_manager is
9
10     signal anodes_switching : std_logic_vector(7 downto 0) := (others => '0');
11
12     begin
13
14     anodes <= not anodes_switching OR not enable_digit;
15
16     anodes_process: process(counter)
17     begin
18         --a seconda del valore di counter le cifre si illuminano una alla volta da destra a sinistra
19         case counter is
20             when "000" =>
21                 anodes_switching <= "00000001";
22             when "001" =>

```

```

23     anodes_switching <= "00000010";
24 when "010" =>
25     anodes_switching <= "00000100";
26 when "011" =>
27     anodes_switching <= "00001000";
28 when "100" =>
29     anodes_switching <= "00010000";
30 when "101" =>
31     anodes_switching <= "00100000";
32 when "110" =>
33     anodes_switching <= "01000000";
34 when "111" =>
35     anodes_switching <= "10000000";
36 when others =>
37     anodes_switching <= (others => '0');
38 end case;
39
40 end process;
41 end Behavioral;

```

Abbiamo un process che al variare del valore di un contatore in ingresso andrà ad attivare un anodo per volta. Vediamo adesso il pilotaggio dei catodi.

```

1  entity cathodes_manager is
2      Port ( counter : in  STD_LOGIC_VECTOR (2 downto 0);
3            value : in  STD_LOGIC_VECTOR (31 downto 0); —dato di mostrare sugli 8 display
4            dots : in  STD_LOGIC_VECTOR (7 downto 0); —configurazione punti da accendere
5            cathodes : out STD_LOGIC_VECTOR (7 downto 0));
6  end cathodes_manager;
7
8  architecture Behavioral of cathodes_manager is
9
10     constant zero : std_logic_vector(6 downto 0) := "1000000";
11     constant one : std_logic_vector(6 downto 0) := "1111001";
12     constant two : std_logic_vector(6 downto 0) := "0100100";
13     constant three : std_logic_vector(6 downto 0) := "0110000";
14     constant four : std_logic_vector(6 downto 0) := "0011001";
15     constant five : std_logic_vector(6 downto 0) := "0010010";
16     constant six : std_logic_vector(6 downto 0) := "0000010";
17     constant seven : std_logic_vector(6 downto 0) := "1111000";
18     constant eight : std_logic_vector(6 downto 0) := "0000000";
19     constant nine : std_logic_vector(6 downto 0) := "0010000";
20     constant a : std_logic_vector(6 downto 0) := "0001000";
21     constant b : std_logic_vector(6 downto 0) := "0000011";
22     constant c : std_logic_vector(6 downto 0) := "1000110";
23     constant d : std_logic_vector(6 downto 0) := "0100001";
24     constant e : std_logic_vector(6 downto 0) := "0000110";
25     constant f : std_logic_vector(6 downto 0) := "0001110";
26
27     alias digit_0 is value (3 downto 0);
28     alias digit_1 is value (7 downto 4);
29     alias digit_2 is value (11 downto 8);
30     alias digit_3 is value (15 downto 12);
31     alias digit_4 is value (19 downto 16);
32     alias digit_5 is value (23 downto 20);
33     alias digit_6 is value (27 downto 24);
34     alias digit_7 is value (31 downto 28);
35
36     signal cathodes_for_digit : std_logic_vector(6 downto 0) := (others => '0');
37     signal nibble : std_logic_vector(3 downto 0) := (others => '0');
38     signal dot : std_logic := '0'; —stabilisce se il punto relativo alla cifra visualizzata deve essere acceso o
39                                     spento
40                                     —nota: dot=1 significa che deve essere acceso, ma il segnale deve essere negato
41                                     per andare sui catodi
42
43     begin
44         — questo processo multiplexa le cifre da mostrare
45         digit_switching: process(counter)
46         begin
47             case counter is
48                 when "000" =>
49                     nibble <= digit_0;
50                     dot <= dots(0);
51                 when "001" =>
52                     nibble <= digit_1;
53                     dot <= dots(1);
54                 when "010" =>
55                     nibble <= digit_2;
56                     dot <= dots(2);
57                 when "011" =>
58                     nibble <= digit_3;
59                     dot <= dots(3);
60                 when "100" =>
61                     nibble <= digit_4;
62                     dot <= dots(4);
63                 when "101" =>
64                     nibble <= digit_5;
65                     dot <= dots(5);
66                 when "110" =>
67                     nibble <= digit_6;

```

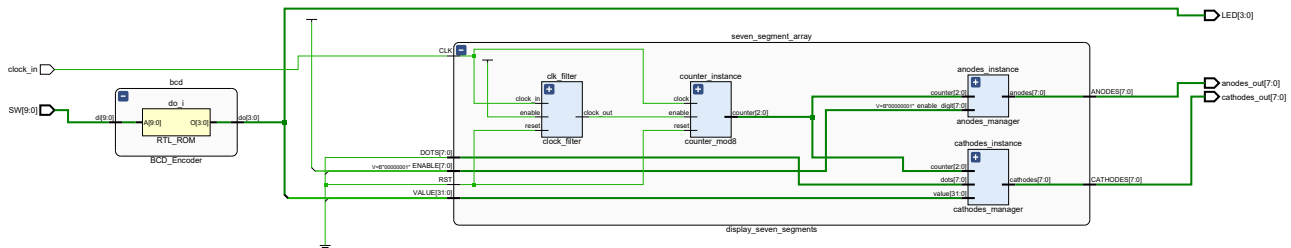
```

68     dot <= dots(6);
69   when "111" =>
70     nibble <= digit_7;
71     dot <= dots(7);
72   when others =>
73     nibble <= (others => '0');
74     dot <= '0';
75   end case;
76 end process;
77
78 seven_segment_decoder_process: process(nibble)
79   begin
80     case nibble is
81       when "0000" => cathodes_for_digit <= zero;
82       when "0001" => cathodes_for_digit <= one;
83       when "0010" => cathodes_for_digit <= two;
84       when "0011" => cathodes_for_digit <= three;
85       when "0100" => cathodes_for_digit <= four;
86       when "0101" => cathodes_for_digit <= five;
87       when "0110" => cathodes_for_digit <= six;
88       when "0111" => cathodes_for_digit <= seven;
89       when "1000" => cathodes_for_digit <= eight;
90       when "1001" => cathodes_for_digit <= nine;
91       when "1010" => cathodes_for_digit <= a;
92       when "1011" => cathodes_for_digit <= b;
93       when "1100" => cathodes_for_digit <= c;
94       when "1101" => cathodes_for_digit <= d;
95       when "1110" => cathodes_for_digit <= e;
96       when "1111" => cathodes_for_digit <= f;
97     when others => cathodes_for_digit <= (others => '0');
98   end case;
99   end process seven_segment_decoder_process;
100
101 cathodes <= (not dot)&cathodes_for_digit;
102
103 end Behavioral;

```

Abbiamo un primo process con il contatore nella sensitivity list. Il segnale nibble assume il valore dei bit in ingresso relativi al display indicati dal contatore. Il secondo process è sensibile a nibble e si occupa di attivare i segmenti relativi alla cifra esadecimale in ingresso.

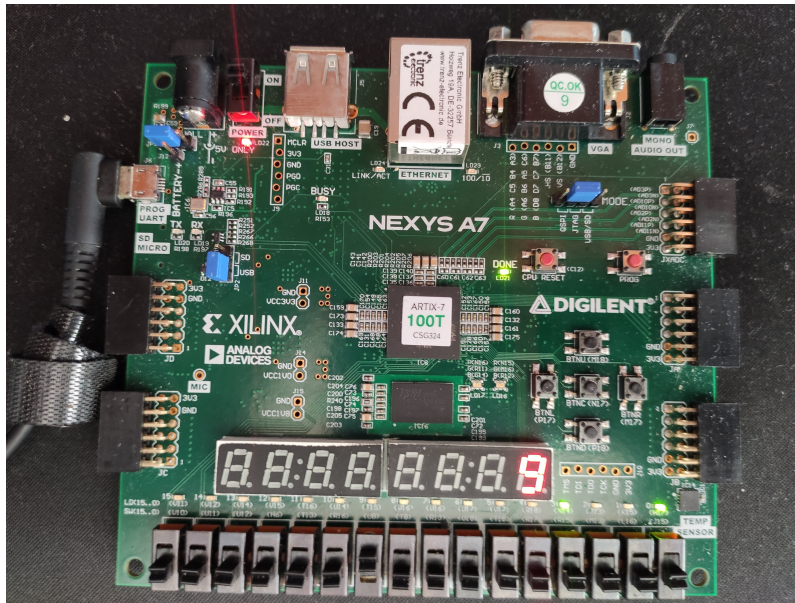
2.3.2 Schematici



I catodi e gli anodi come detto precedentemente sono connessi a un contatore. Tale contatore effettua il refresh dei segmenti a una frequenza sufficiente da essere invisibile all'occhio umano. Il contatore riceve in ingresso un segnale di clock diviso in frequenza in modo opportuno. L'uscita del BCD è connessa oltre che ai LED anche al blocco del display a sette segmenti. Anodi e catodi sono stati associati mediante constraints.

2.3.3 Sintesi su board FPGA

Una volta stabiliti tutti i constraints e implementato correttamente il display il componente sviluppato funziona senza particolari problemi sulla board. Alzando uno degli switch viene visualizzato il numero dello switch attivo e viene visualizzata con i LED la rappresentazione in binario di tale numero.



Capitolo 3

Riconoscitore di sequenza

Esercizio 3.1

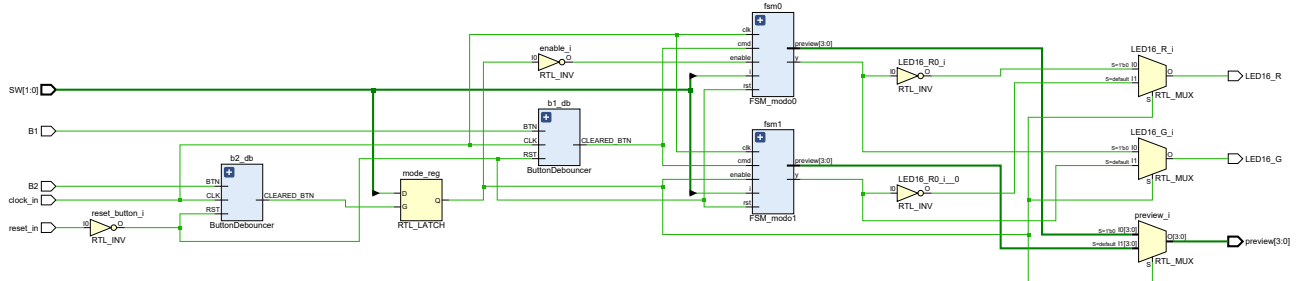
Progettare, implementare in VHDL e testare mediante simulazione una macchina in grado di riconoscere la sequenza 1001. La macchina prende in ingresso un segnale binario i che rappresenta il dato, un segnale A di tempificazione e un segnale M di modo, che ne disciplina il funzionamento, e fornisce un'uscita Y alta quando la sequenza viene riconosciuta. In particolare,

- se $M=0$, la macchina valuta i bit seriali in ingresso a gruppi di 4,
- se $M=1$, la macchina valuta i bit seriali in ingresso uno alla volta, tornando allo stato iniziale ogni volta che la sequenza viene correttamente riconosciuta.

Esercizio 3.2

Sintetizzare e implementare su board la rete sviluppata al punto precedente, utilizzando uno switch $S1$ per codificare l'input i e uno switch $S2$ per codificare il modo M , in combinazione con due bottoni $B1$ e $B2$ utilizzati rispettivamente per acquisire l'input da $S1$ e $S2$ in sincronismo con il segnale di tempificazione A , che deve essere ottenuto a partire dal clock della board. Infine, l'uscita Y può essere codificata utilizzando un led.

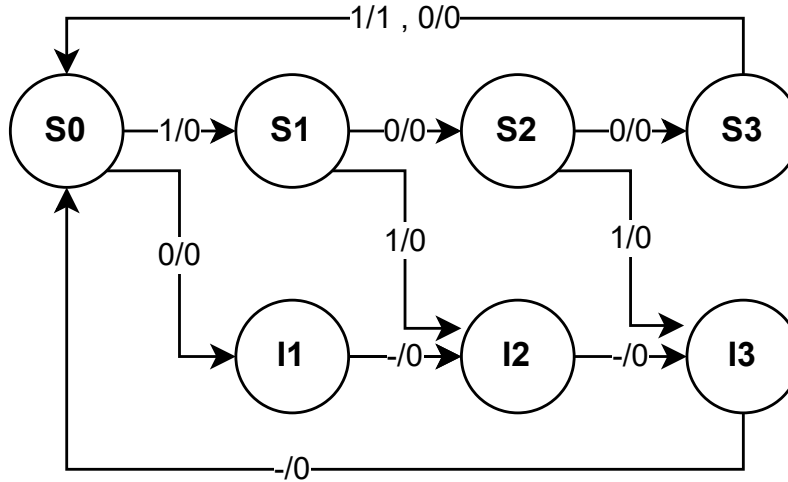
3.1 Schematici



Visto che a livello implementativo il secondo punto aggiunge solamente due button debouncer e la definizione dei constraint si è deciso di sviluppare entrambi in un sol colpo. Le due macchine a stati finiti sfruttano un clock e hanno in ingresso il valore dello switch e il pulsante pulito dal debouncer, in uscita vi è un segnale y di sequenza confermata che poi viene mostrato in uscita con i LED RGB. In uscita vi è anche un vettore di segnali $preview$ per mostrare coi LED inferiori gli ultimi quattro input.

3.2 Codice

Le due modalità di riconoscimento sono state implementate come automi a stati finiti, in particolare come macchine di Mealy la cui uscita dipende dalle transizioni. Di seguito l'automato del modo 0 e la relativa implementazione in VHDL.



```

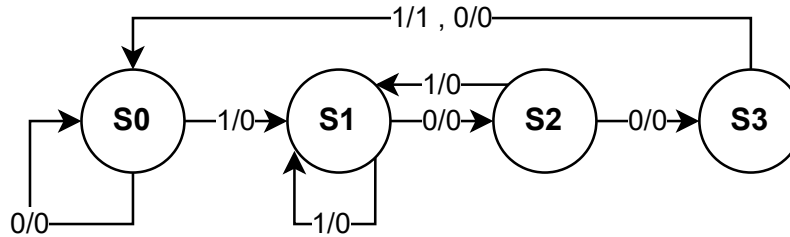
1  type stato is (S0,S1,S2,S3,I1,I2,I3);
2  signal stato_corrente: stato := S0;
3  signal tmp: std_logic_vector(3 downto 0);
4  begin
5
6  stato_uscita: process (clk, rst)
7  variable tmp: std_logic_vector(3 downto 0) := "0000";
8  begin
9      if (rst = '1') then
10         y <= '0';
11         stato_corrente <= S0;
12     elsif (rising_edge(clk) and enable = '1') then
13         if cmd = '1' then
14             tmp := tmp(2 downto 0) & i;
15             case stato_corrente is
16                 when S0 =>
17                     if (i = '1') then
18                         stato_corrente <= S1;
19                         y <= '0';
20                     else
21                         stato_corrente <= I1;
22                         y <= '0';
23                     end if;
24                 when S1 =>
25                     if (i = '0') then
26                         stato_corrente <= S2;
27                         y <= '0';
28                     else
29                         stato_corrente <= I2;
30                         y <= '0';
31                     end if;
32                 when S2 =>
33                     if (i = '0') then
34                         stato_corrente <= S3;
35                         y <= '0';
36                     else
37                         stato_corrente <= I3;
38                         y <= '0';
39                     end if;
40                 when S3 =>
41                     if (i = '1') then
42                         stato_corrente <= S0;
43                         y <= '1';
44                     else
45                         stato_corrente <= S0;
46                         y <= '0';
47                     end if;
48                 when I1 =>
49                     stato_corrente <= I2;
50                     y <= '0';
51                 when I2 =>
52                     stato_corrente <= I3;
53                     y <= '0';
54             end case;
55             tmp := "0000";
56         end if;
57     end if;
58 end process;
59 
```

```

54     y <= '0';
55     when I3 =>
56         stato_corrente <= S0;
57         y <= '0';
58         tmp := "0000";
59     end case;
60 end if;
61 end if;
62 preview <= tmp;
63 end process;

```

E l'automa e il codice per il modo 1. Da notare che da specifica l'automa una volta riconosciuta la sequenza torna allo stato iniziale.

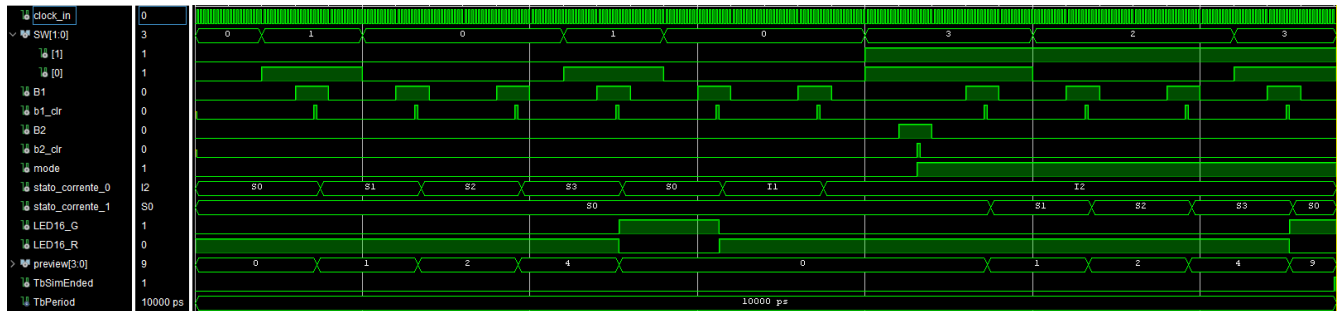


```

1  type stato is (S0,S1,S2,S3);
2  signal stato_corrente: stato := S0;
3  signal tmp: std_logic_vector(3 downto 0) := "0000";
4  begin
5  preview <= tmp;
6  stato_uscita: process (clk, rst)
7  begin
8      if (rst = '1') then
9          y <= '0';
10         stato_corrente <= S0;
11     elsif (rising_edge(clk) and enable = '1') then
12         if cmd = '1' then
13             tmp <= tmp(2 downto 0) & i; --In realt basterebbe verificare che questo sia uguale a "1001" ma per
14                                     seguire lo spirito dell'esercizio lo faccio con un automa.
15         case stato_corrente is
16             when S0 =>
17                 if (i = '1') then
18                     stato_corrente <= S1;
19                     y <= '0';
20                 else
21                     stato_corrente <= S0;
22                     y <= '0';
23                 end if;
24             when S1 =>
25                 if (i = '0') then
26                     stato_corrente <= S2;
27                     y <= '0';
28                 else
29                     stato_corrente <= S1;
30                     y <= '0';
31                 end if;
32             when S2 =>
33                 if (i = '0') then
34                     stato_corrente <= S3;
35                     y <= '0';
36                 else
37                     stato_corrente <= S1;
38                     y <= '0';
39                 end if;
40             when S3 =>
41                 if (i = '1') then
42                     stato_corrente <= S0;
43                     y <= '1';
44                 else
45                     stato_corrente <= S0;
46                     y <= '0';
47                 end if;
48             end case;
49         end if;
50     end process;

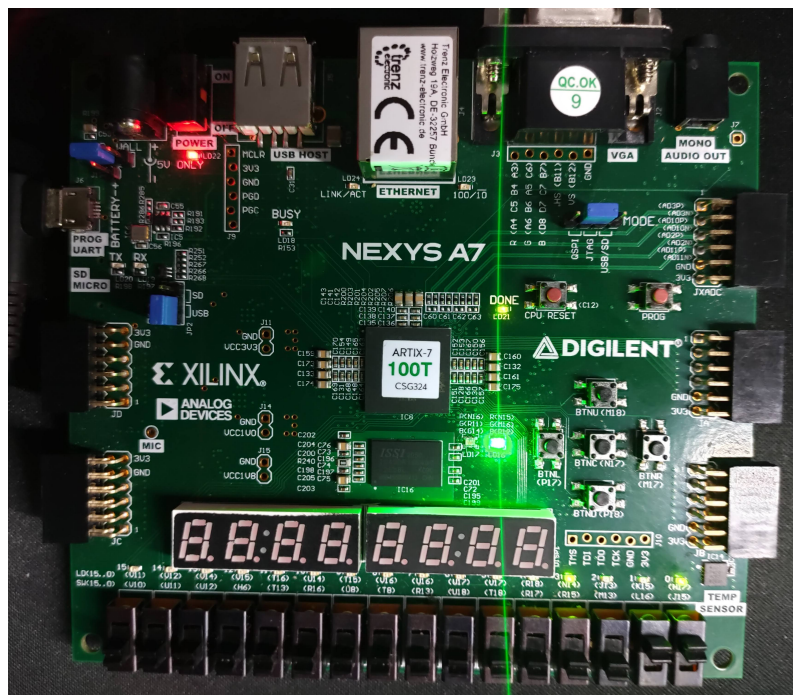
```

3.3 Simulazione



In simulazione si può vedere il comportamento dei due modi di funzionamento in caso di sequenza corretta. Per il modo 0 vengono mostrati anche un paio di stati in cui la sequenza non è valida che servono per attendere l'inserimento dell'intero gruppo di 4 bit.

3.4 Sintesi su board FPGA



Capitolo 4

Shift Register

Esercizio 4.1 *Progettare, implementare in VHDL e testare mediante simulazione un registro a scorrimento di N bit in grado di shiftare a destra o a sinistra di un numero Y variabile di posizioni a seconda di una opportuna selezione. Il componente deve essere realizzato utilizzando sia un a) approccio comportamentale sia un b) approccio strutturale. Nota: il numero di bit del registro X e i valori che può assumere il parametro Y possono essere scelti dallo studente (ad es. $X=8$ e $Y=1,2$).*

4.1 Approccio comportamentale

4.1.1 Codice

Vengono scelti $X = 8$ (il componente è stato realizzato con X generico ma verrà simulato pari ad 8) e $Y = 1, 2$. Per realizzare lo shift register viene usata una variabile interna al processo sensibile sul clock. Con dei cicli for vengono effettuati gli shift (potevano anche essere usate delle più semplici assegnazioni volendo), se lo shift avviene verso sinistra viene aggiunto uno 0 nell'ultimo registro, se verso destra viene inserito il valore in ingresso nel primo registro. Un segnale `double_shift` permette di ripetere queste operazioni una seconda volta, nel caso di un doppio shift a destra (sinistra) si suppone che l'ingresso (uno zero) venga copiato due volte nei primi (ultimi) due registri.

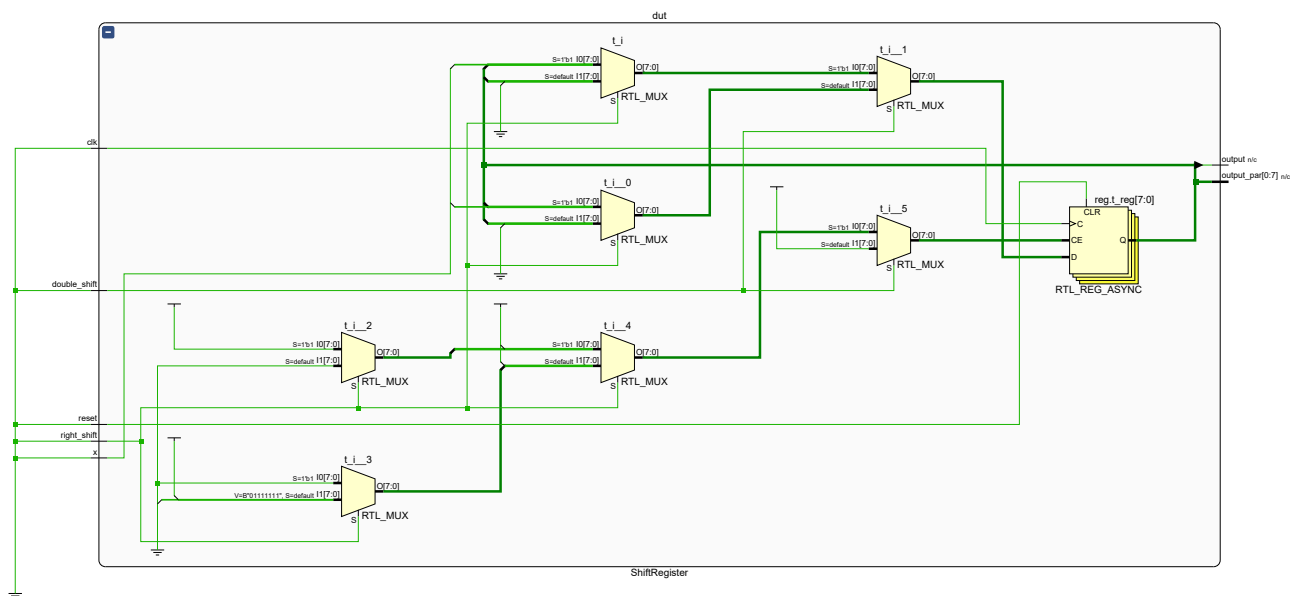
```
1  entity ShiftRegister is
2      generic (n: INTEGER := 8);
3      Port (
4          clk          : in STD_LOGIC;
5          reset        : in STD_LOGIC;
6          x            : in STD_LOGIC;
7          double_shift  : in STD_LOGIC;
8          right_shift   : in STD_LOGIC; — '1' for right shift, '0' for left shift
9          output        : out STD_LOGIC;
10         output_par    : out STD_LOGIC_VECTOR(0 to n-1)
11     );
12 end ShiftRegister;
13
14 architecture Behavioral of ShiftRegister is
15 begin
16     reg: process (clk, reset)
17         variable t: STD_LOGIC_VECTOR(0 to n-1);
18     begin
19         if (reset = '1') then
20             t := (others => '0');
21         elsif rising_edge(clk) then
22
23             if right_shift = '1' then
24                 for i in 1 to n-1 loop
25                     t(n-i) := t(n-i-1);
26                 end loop;
27                 t(0) := x;
28             else
29                 for i in 0 to n-2 loop
```

```

30         t(i) := t(i+1);
31     end loop;
32     t(n-1) := '0';
33 end if;
34
35 if double_shift = '1' then
36     if right_shift = '1' then
37         for i in 1 to n-1 loop
38             t(n-i) := t(n-i-1);
39         end loop;
40         t(0) := x;
41     else
42         for i in 0 to n-2 loop
43             t(i) := t(i+1);
44         end loop;
45         t(n-1) := '0';
46     end if;
47 end if;
48
49 output_par <= t;
50 output <= t(n-1);
51 end process;
52 end Behavioral;

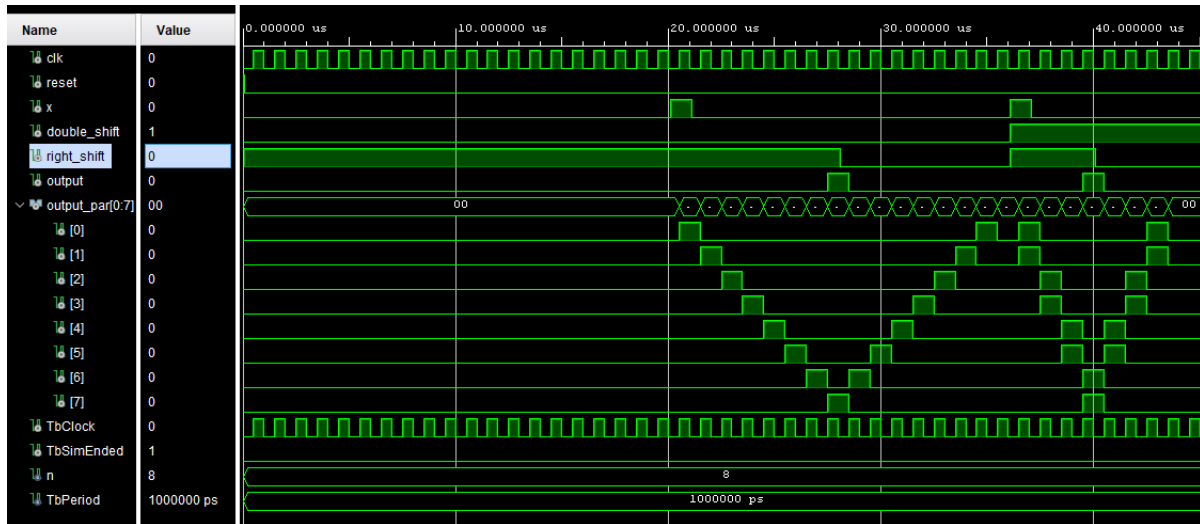
```

4.1.2 Schematici



Il design viene sintetizzato da Vivado come un insieme di multiplexer regolati dai segnali di `shift_right` e `double_shift` e da un banco di registri che formano la componente di memoria.

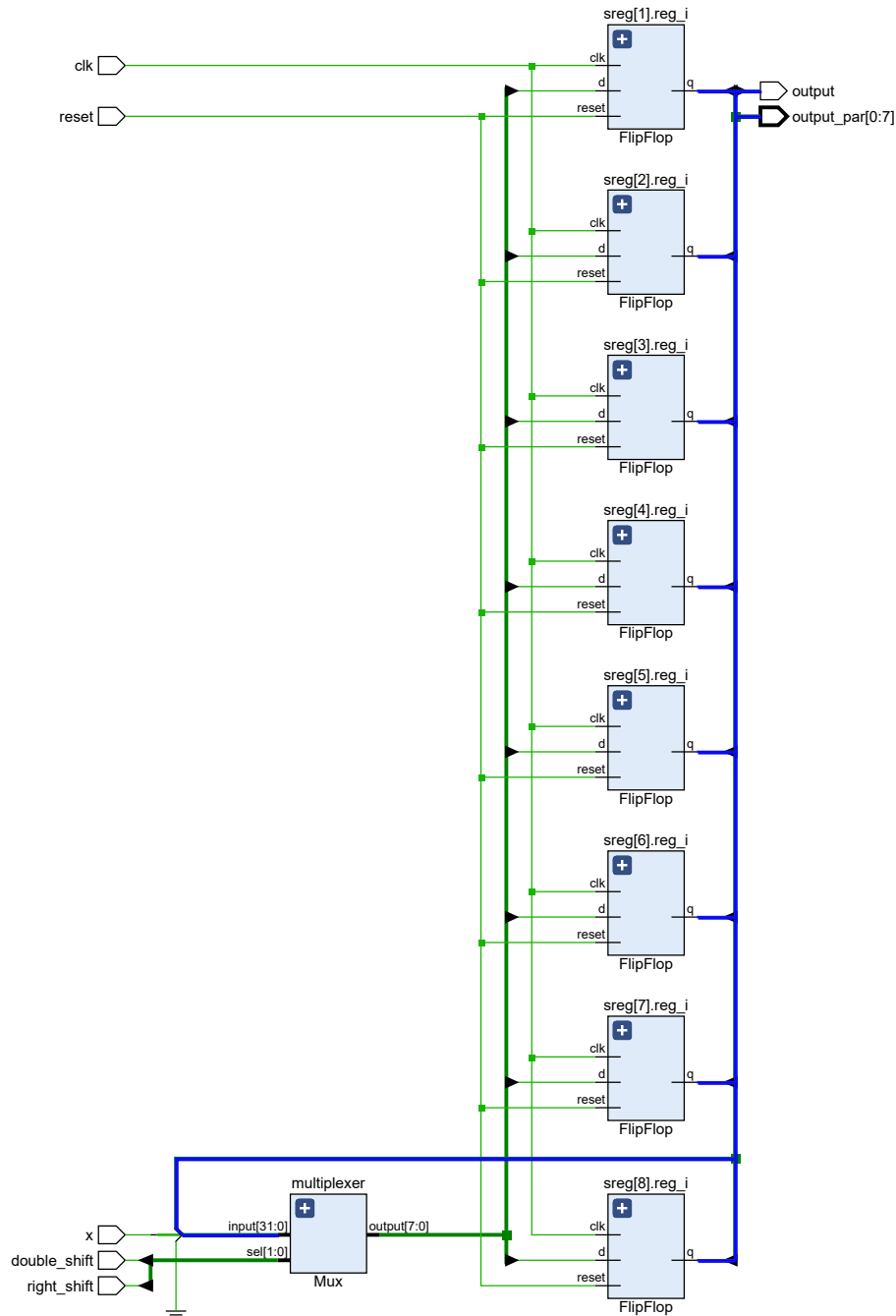
4.1.3 Simulazione



In simulazione viene provato lo shift a destra e poi a sinistra di un singolo bit alto dato in ingresso. La stessa prova viene rifatta per un double shift.

4.2 Approccio strutturale

4.2.1 Schematici



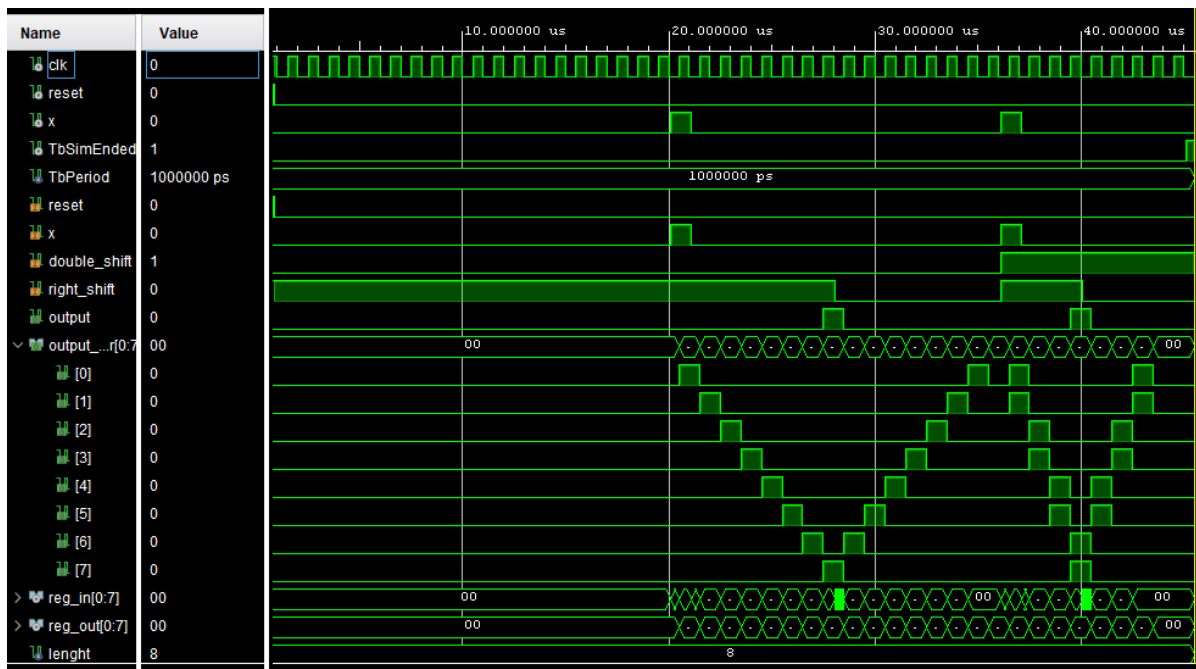
Con un costrutto generato viene creato un banco di $n = 8$ registri. Le uscite di tali registri vengono opportunamente connesse a un multiplexer che in ingresso avrà 4 gruppi da 8 bit a seconda del tipo di shift effettuato. I segnali di selezione del mux sono quelli di `right_shift` e `double_shift`. Si allega una piccola porzione del codice che mostra nel dettaglio come sono connessi registri e multiplexer

```

1  multiplexer: Mux generic map(N => 4, M => lenght)
2  port map(
3    input => x & x & reg_out(0 to lenght-3) & —right_shift = 1, double_shift = 1
4    x & reg_out(0 to lenght-2) & —right_shift = 1, double_shift = 0
5    reg_out(2 to lenght-1) & "00" & —right_shift = 0, double_shift = 1
6    reg_out(1 to lenght-1) & '0', —right_shift = 0, double_shift = 0
7    sel => right_shift & double_shift,
8    output => reg_in
9  );
10
11 sreg : for i in 1 to lenght generate
12   reg_i: FlipFlop port map(
13     clk => clk,
14     reset => reset,
15     d => reg_in(lenght-i),
16     q => reg_out(lenght-i)
17   );
18 end generate sreg;

```

4.2.2 Simulazione



Viene riutilizzato lo stesso testbench usato per l'approccio comportamentale verificando che i risultati coincidano.

Capitolo 5

Cronometro

5.1 Esercizio 5.1

Progettare, implementare in VHDL e testare mediante simulazione un cronometro, in grado di scandire secondi, minuti e ore a partire da una base dei tempi prefissata (es. si consideri il clock a disposizione sulla board). Il progetto deve prevedere la possibilità di inizializzare il cronometro con un valore iniziale, sempre espresso in termini di ore, minuti e secondi, mediante un opportuno ingresso di set, e deve prevedere un ingresso di reset per azzerare il tempo. Il componente deve essere realizzato utilizzando un approccio strutturale, collegando opportunamente dei contatori secondo uno schema a scelta.

5.1.1 Unità operativa - Orologio

5.1.1.1 Contatore

Partiamo dal blocco elementare di conteggio utilizzato in questo progetto, si tratta di un contatore modulo N con reset e load asincrono realizzato con i generics per poter specificare il modulo e il numero di bit favorendone il riuso. In uscita abbiamo sia il valore corrente del conteggio che un segnale y che assume il livello logico alto quando si arriva a contare N. In ingresso abbiamo il clock, un segnale di reset, di enable, di load e il valore a cui viene settato il contatore. Il segnale indicato come clock può in genere essere qualunque tipo di segnale di conteggio in questo caso. L'uscita del contatore quando questo è in load segue il dato in ingresso.

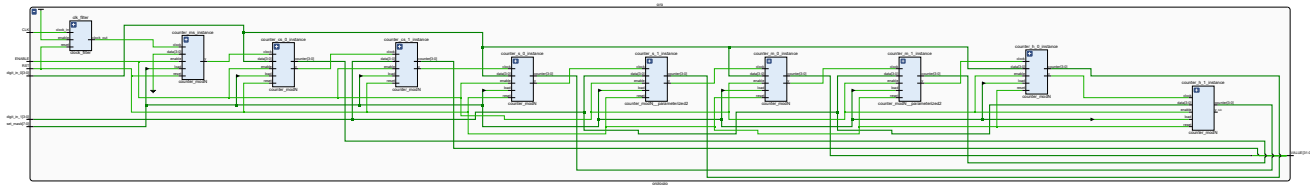
```
1  entity counter_modN is
2      generic(
3          n_bit: integer :=2;
4          n : integer := 0
5      );
6      Port ( clock : in STD_LOGIC;
7            reset  : in STD_LOGIC;
8            enable : in STD_LOGIC;
9            load   : in STD_LOGIC;
10           data : in STD_LOGIC_VECTOR(n_bit-1 downto 0);
11           y : out std_logic;
12           counter : out STD_LOGIC_VECTOR (n_bit-1 downto 0));
13 end counter_modN;
14
15 architecture Behavioral of counter_modN is
16
17     signal c: std_logic_vector (n_bit-1 downto 0) := (others => '0');
18     signal ty: std_logic;
19
20 begin
21
22
23     counter_process: process(clock , reset , load)
24 begin
```

```

25 if reset = '1' then
26   c <= (others => '0');
27   ty <= '0';
28 elseif (load = '1') then
29   if unsigned(data) >= n then
30     c <= std_logic_vector(to_unsigned(n-1,n_bit));
31   else
32     c <= data;
33   end if;
34 elseif (rising_edge(clock)) then
35   if enable = '1' then
36     if (unsigned(c) = (n-1)) then
37       c <= (others => '0');
38       ty <= '1';
39     else
40       c <= std_logic_vector(unsigned(c)+1);
41       ty <= '0';
42     end if;
43   end if;
44 end if;
45 end process;
46 counter <= c;
47 y <= ty;
48
49 end Behavioral;

```

5.1.1.2 Schematici



I contatori sono stati usati sfruttando un approccio strutturale per costruire un componente chiamato Orologio. Ogni contatore ha N pari al valore massimo di una cifra dell'orario. Ad esempio il formato hh:mm:ss ha due contatori modulo 10 per le ore, un contatore modulo 6 e uno modulo 10 per ore e secondi. Questa scelta è stata fatta anche in ottica del successivo punto dell'esercizio che prevede il collegamento ai display della board e permette una notevole semplificazione implementativa consentendo il collegamento diretto dell'uscita dei contatori ai rispettivi display.

Si è scelto di aggiungere anche dei contatori dei decimi e centesimi di secondo (visualizzati sul display) e dei millesimi di secondo (non mostrato sul display) per avere una risoluzione del conteggio in uscita più elevata.

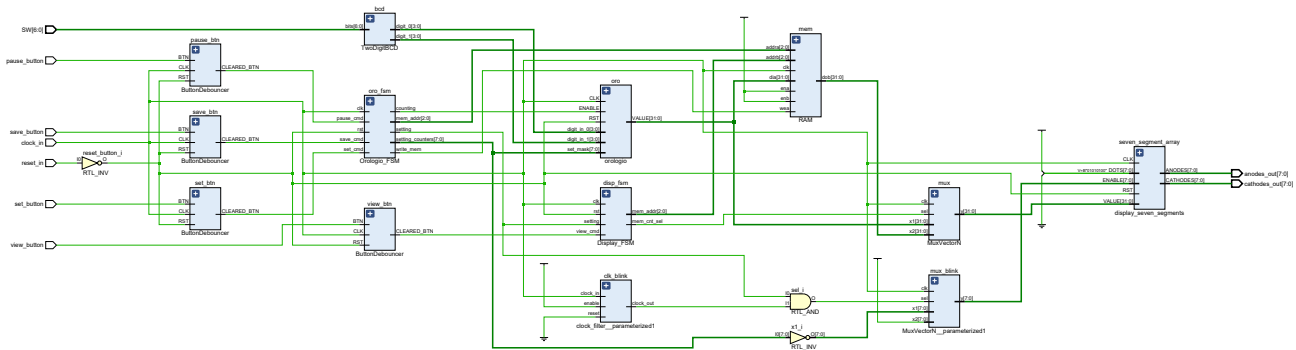
In ingresso abbiamo due coppie da 4 bit per le cifre da impostare e una set mask che serve a indicare su quali contatori memorizzare le cifre.

I contatori sono connessi in cascata ossia il segnale y di uno è connesso al clock del contatore successivo. A monte di ciò è presente un divisore di frequenza.

reset. Si utilizzi una codifica a scelta dello studente per la visualizzazione dell'orario sui display (esadecimale o decimale).

Esercizio 5.3 Estendere il componente sviluppato ai punti precedenti in modo che sia in grado di acquisire e memorizzare internamente fino ad un numero N di intertempi in corrispondenza di un ingresso di stop. Opzionalmente, il componente può prevedere una modalità di visualizzazione in cui, alla pressione di un bottone, vengano visualizzati sui display gli intertempi memorizzati (uno per ogni pressione).

5.2.1 Schematici



Per svolgere questi due esercizi sono state implementate due macchine a stati finite, una che pilotasse il componente Orologio e una che pilotasse il display. A seconda dello stato in cui si trova il cronometro (visualizzazione, conteggio, pausa, inserimento) i due componenti funzionano in modo differente e per semplificare il problema si è scelto di avere due unità di controllo separate che utilizzano delle semplici macchine a stati finiti. Viene inoltre implementata una semplice RAM per la memorizzazione degli intertempi. Una possibile alternativa per questo componente sarebbe stata l'utilizzo di uno shift register e di un multiplexer per visualizzare le varie slice dell'uscita parallela.

Un convertitore binario a BCD a due cifre viene usato per ricevere gli input dagli switch.

Gli ingressi ricevuti dai pulsanti hanno tutti un debouncer.

È presente lo stesso componente che pilota il display visto precedentemente.

Un multiplexer e un divisore di frequenza permettono il lampeggio del display durante il caricamento dei contatori. Un altro multiplexer come vedremo permette di selezionare l'uscita della memoria o quella dei contatori.

5.2.2 Convertitore Binario-BCD

Il convertitore funziona diversamente dall'encoder BCD visto precedentemente. In questo caso consideriamo un numero decimale, ad esempio $43_{(10)}$ quello che vogliamo fare è convertire la sua rappresentazione binaria 101011 in 0100 e 0011 ossia la rappresentazione binaria delle cifre (BCD) di $43_{(10)}$. La prima cifra si ottiene come resto della divisione per 10, la seconda cifra si ottiene dividendo per 10 ed effettuando nuovamente l'operazione di modulo.

```

1 entity TwoDigitBCD is
2   Port ( bits : in STD.LOGIC.VECTOR (6 downto 0);
3         digit_0 : out STD.LOGIC.VECTOR (3 downto 0);
4         digit_1 : out STD.LOGIC.VECTOR (3 downto 0));

```

```

5 end TwoDigitBCD;
6
7 architecture Behavioral of TwoDigitBCD is
8 begin
9   bcd: process(bits)
10    variable temp: unsigned(6 downto 0);
11    begin
12      temp := unsigned(bits);
13      if temp > 99 then
14        temp := to_unsigned(99,7);
15      end if;
16
17      digit_0 <= std_logic_vector(to_unsigned(to_integer(temp) mod 10,4));
18      temp := temp/10;
19      digit_1 <= std_logic_vector(to_unsigned(to_integer(temp) mod 10,4));
20
21    end process;
22 end Behavioral;

```

5.2.3 RAM

Per realizzare la RAM si è fatto riferimento al modello d'esempio di Xilinx¹. Abbiamo una RAM con due porte, una di scrittura (A) e una di lettura (B). I segnali sono quelli di write enable sulla porta A, di indirizzo A e indirizzo B, data in A e data out B.

```

1  entity RAM is
2  port(
3
4    clk      : in  std_logic;
5    ena      : in  std_logic;
6    enb      : in  std_logic;
7    wea      : in  std_logic;
8    addra    : in  std_logic_vector(2 downto 0);
9    addrb    : in  std_logic_vector(2 downto 0);
10   dia      : in  std_logic_vector(31 downto 0);
11   dob      : out std_logic_vector(31 downto 0)
12 );
13
14 end RAM;
15
16 architecture syn of RAM is
17 type ram_type is array (7 downto 0) of std_logic_vector(31 downto 0);
18 shared variable RAM : ram_type;
19
20 begin
21
22 process (clk)
23 begin
24   if clk'event and clk = '1' then
25     if ena = '1' then
26       if wea = '1' then
27         RAM(conv_integer(addra)) := dia;
28       end if;
29     end if;
30   end if;
31 end process;
32
33 process (clk)
34 begin
35   if clk'event and clk = '1' then
36     if enb = '1' then
37       dob <= RAM(conv_integer(addrb));
38     end if;
39   end if;
40 end process;
41
42 end syn;

```

5.2.4 Unità di Controllo Display

L'unità di controllo del display è una semplice macchina a stati finiti, riceve i segnali di setting (ossia quando è in corso il caricamento dei registri del cronometro) e il comando di view che provengono da dei pulsanti. In uscita abbiamo l'indirizzo della memoria da visualizzare e il segnale

¹Simple Dual-Port RAM. URL: <https://docs.xilinx.com/r/en-US/ug901-vivado-synthesis/Simple-Dual-Port-Block-RAM-with-Single-Clock-VHDL>.

mem_cnt_sel che funziona come selezione per un multiplexer che cambia l'ingresso del display dalla memoria ai contatori dell'orologio.

5.2.4.1 Codice

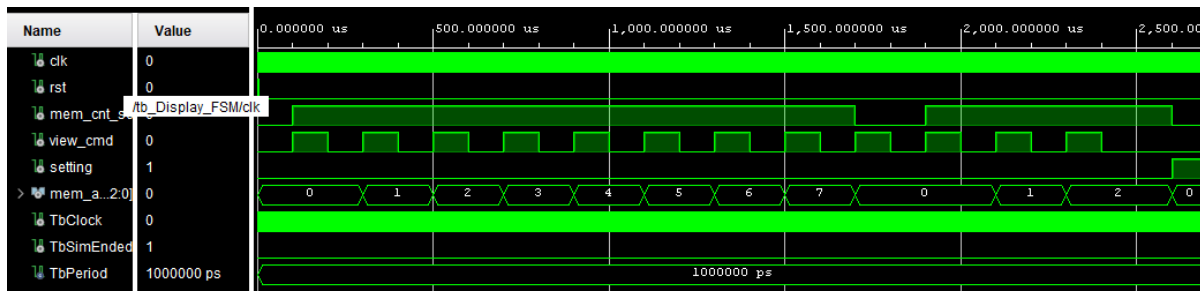
```

1  entity Display_FSM is
2      Port (
3          clk : in  STD_LOGIC;
4          rst : in  STD_LOGIC;
5          mem_cnt_sel : out std_logic;
6          view_cmd : in  std_logic;
7          setting : in  std_logic;
8          mem_addr : out std_logic_vector(2 downto 0)
9      );
10 end Display_FSM;
11
12
13 architecture Behavioral of Display_FSM is
14     type state is (counters, mem);
15     signal current_state : state;
16     signal next_state : state := counters;
17     signal mem_counter : std_logic_vector(2 downto 0);
18     signal mem_cnt : std_logic := '0';
19
20     begin
21
22     reg: process(clk, rst)
23     begin
24
25         if rising_edge(clk) then
26             current_state <= next_state;
27         end if;
28     end process reg;
29
30     comb: process(view_cmd, setting, rst)
31     begin
32         if (rst = '1') then
33             next_state <= counters;
34             mem_cnt <= '0';
35             mem_counter <= "000";
36         else
37             if (setting = '1') then
38                 next_state <= counters;
39                 mem_cnt <= '0';
40                 mem_counter <= "000";
41             elsif view_cmd = '1' then
42                 case current_state is
43                     when counters => next_state <= mem;
44                                     mem_cnt <= '1';
45
46
47                     when mem => if (mem_counter = "111") then
48                                     mem_counter <= "000";
49                                     next_state <= counters;
50                                     mem_cnt <= '0';
51                                 else
52                                     mem_counter <= std_logic_vector(unsigned(mem_counter) + 1);
53                                     next_state <= mem;
54                                 end if;
55                 end case;
56             end if;
57         end if;
58     end process comb;
59     mem_cnt_sel <= mem_cnt;
60     mem_addr <= mem_counter;
61 end Behavioral;
62

```

Ad ogni pressione del pulsante view questa macchina a stati finiti itera sugli 8 registri della memoria e sull'uscita dei contatori del componente orologio. Quando viene ricevuto il comando di setting la visualizzazione viene riportata all'uscita dei contatori.

5.2.4.2 Simulazione



Nella simulazione viene ripetutamente inserito il comando di view e verificato il funzionamento del comando di setting. Viene anche verificato il corretto funzionamento del segnale mem_cnt_sel.

5.2.5 Unità di Controllo Orologio

La FSM che si occupa di controllare l'orologio ha molteplici compiti. Il primo è di abilitare e disabilitare il conteggio quando viene ricevuta la pressione del pulsante di pausa.

Deve anche scandire la bitmask alla ricezione del comando di set per il caricamento dei contatori e infine deve inviare il segnale di write enable e l'indirizzo della locazione di memoria a cui salvare l'intertempo.

```

1  entity Orologio.FSM is
2  Port (
3      clk: in std_logic;
4      rst: in std_logic;
5      pause_cmd: in std_logic;
6      set_cmd: in std_logic;
7      save_cmd: in std_logic;
8      counting: out std_logic := '1';
9      setting: out std_logic := '0';
10     setting_counters: out std_logic_vector(7 downto 0) := "00000000";
11     write_mem: out std_logic;
12     mem_addr: out std_logic_vector(2 downto 0)
13 );
14 end Orologio.FSM;
15
16 architecture Behavioral of Orologio.FSM is
17
18     type state is (running, paused, set_0, set_1, set_2, set_3);
19     signal current_state: state;
20     signal mem_counter : std_logic_vector (2 downto 0) := "111";
21     signal counting_out : std_logic;
22
23 begin
24
25     comb: process(clk, rst)
26     begin
27         if (rst = '1') then
28             counting_out <= '1';
29             setting <= '0';
30             setting_counters <= "00000000";
31             current_state <= running;
32         elsif rising_edge(clk) then
33             if (pause_cmd = '1') then
34                 case current_state is
35                     when running => counting_out <= '0';
36                     current_state <= paused;
37                     when paused => counting_out <= '1';
38                     current_state <= running;
39                     when others =>
40                         end case;
41             end if;
42
43             if (save_cmd = '1') then
44                 if (mem_counter = "111") then
45                     mem_counter <= "000";
46                 else
47                     mem_counter <= std_logic_vector(unsigned(mem_counter) + 1);
48                 end if;
49                 write_mem <= '1';
50             else
51                 write_mem <= '0';
52             end if;
53         end if;
54     end process;
55 end

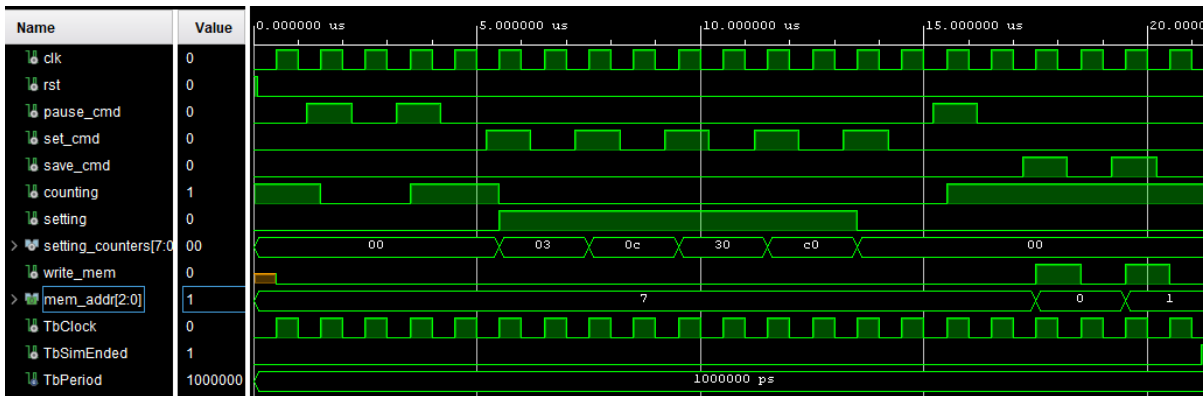
```

```

54
55   if (set_cmd = '1') then
56     case current_state is
57       when running => counting_out <= '0';
58                       current_state <= set_0;
59                       setting <= '1';
60                       setting_counters <= "00000011";
61       when paused => counting_out <= '0';
62                       setting <= '1';
63                       current_state <= set_0;
64                       setting_counters <= "00000011";
65       when set_0 => counting_out <= '0';
66                   current_state <= set_1;
67                   setting_counters <= "00001100";
68       when set_1 => counting_out <= '0';
69                   current_state <= set_2;
70                   setting_counters <= "00110000";
71       when set_2 => counting_out <= '0';
72                   current_state <= set_3;
73                   setting_counters <= "11000000";
74       when set_3 => counting_out <= '0';
75                   current_state <= paused;
76                   setting_counters <= "00000000";
77                   setting <= '0';
78     end case;
79   end if;
80 end if;
81
82 end process comb;
83
84 mem_addr <= mem_counter;
85 counting <= counting_out;
86
87 end Behavioral;

```

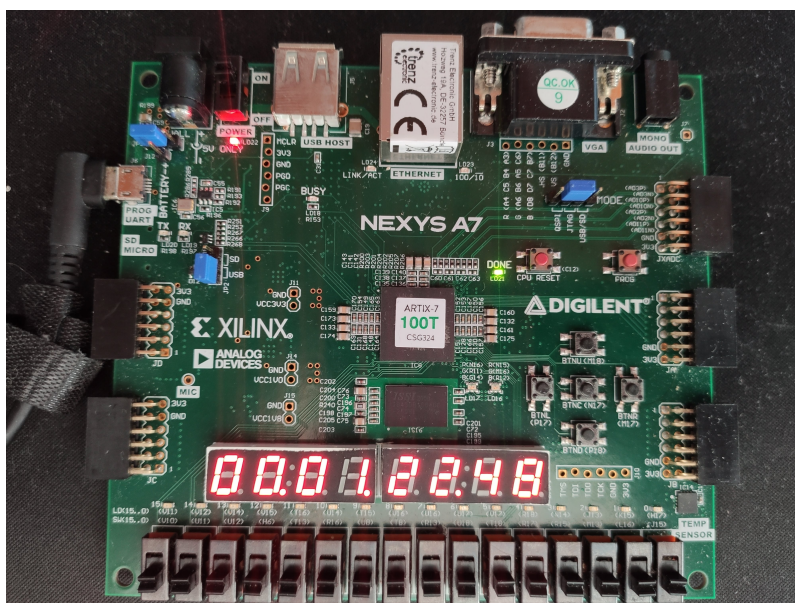
5.2.5.1 Simulazione



In simulazione vengono inviati i segnali di pausa, di setting e di write. Come previsto l'uscita del segnale counting si abbassa alla prima pressione del pulsante di pausa e si alza a quella successiva. Per quanto riguarda il setting dei contatori ad ogni pressione viene attivata una bitmask differente e vengono caricati due contatori per volta che riceveranno come visto prima il loro ingresso da un BCD.

Quando viene premuto il pulsante di write viene attivato il segnale di write_mem e incrementato l'indirizzo della porta A della memoria.

5.2.6 Sintesi su board FPGA



Capitolo 6

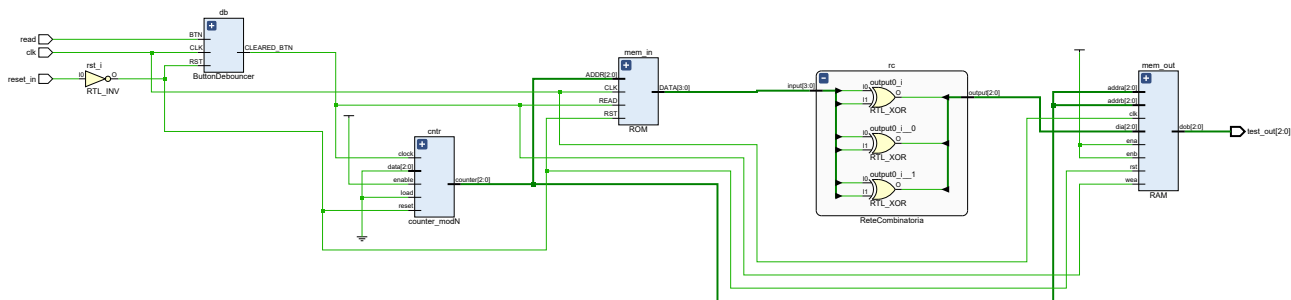
Sistema di Testing

6.1 Esercizio 6

Esercizio 6.1 Progettare, implementare in VHDL e verificare mediante simulazione un sistema in grado di testare in maniera automatica una macchina combinatoria M avente 4 ingressi e 3 uscite binarie sottoponendole N ingressi diversi (si considerino una macchina M e un numero di input N a scelta dello studente). Gli N valori di input per il test devono essere letti da una ROM, in cui essi sono precaricati, in corrispondenza di un segnale read. Le N uscite fornite dalla macchina in corrispondenza di ciascuno degli input devono essere memorizzate in una memoria interna, che deve poter essere svuotata in qualsiasi momento in presenza di un segnale di reset.

Esercizio 6.2 Sintetizzare ed implementare su board il componente sviluppato al punto precedente, utilizzando due bottoni per i segnali di read e reset rispettivamente e i led per la visualizzazione delle uscite della macchina istante per istante.

6.1.1 Schematici



È stato fatto un massiccio utilizzo dei componenti già visti precedentemente, le locazioni in uscita dalla ROM e in ingresso nella RAM vengono scandite da un contatore ed è stato creato un componente che rappresenti la rete combinatoria. Per la fase di implementazione sulla board è stato aggiunto un button debouncer per il segnale di read e l'uscita della RAM, scandita dallo stesso contatore è connessa ai LED della board.

6.1.2 Codice

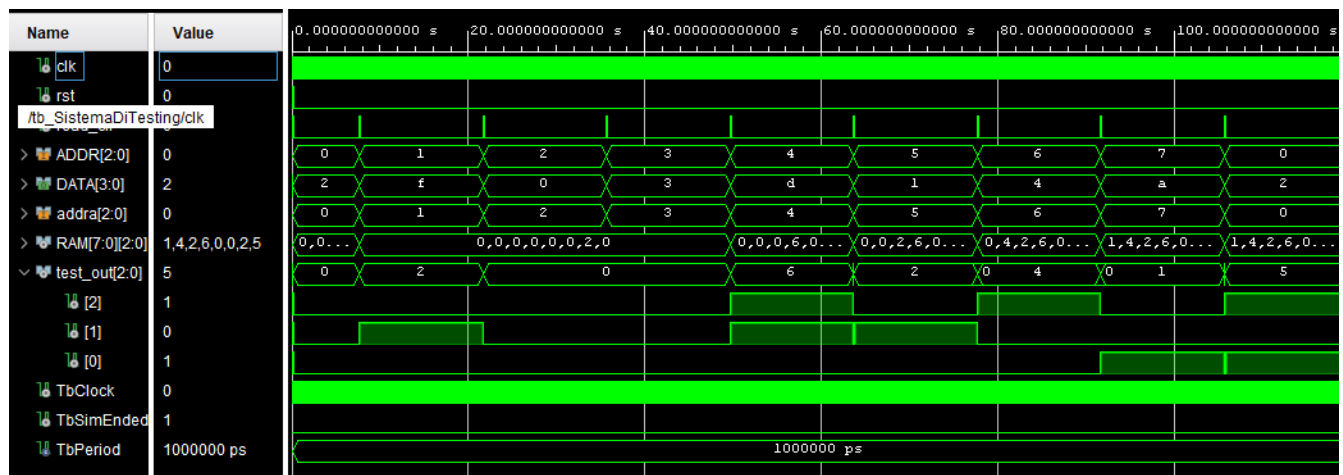
Di seguito si allega il codice della rete combinatoria utilizzata, ogni uscita rappresenta lo XOR tra l'ultimo input e gli altri tre segnali.

```

1  entity ReteCombinatoria is
2      Port ( input : in STD_LOGIC_VECTOR (3 downto 0);
3            output : out STD_LOGIC_VECTOR (2 downto 0));
4  end ReteCombinatoria;
5
6  architecture Dataflow of ReteCombinatoria is
7  begin
8      output <= (input(0) xor input(3)) &
9                (input(1) xor input(3)) &
10               (input(2) xor input(3));
11  end Dataflow;

```

6.1.3 Simulazione

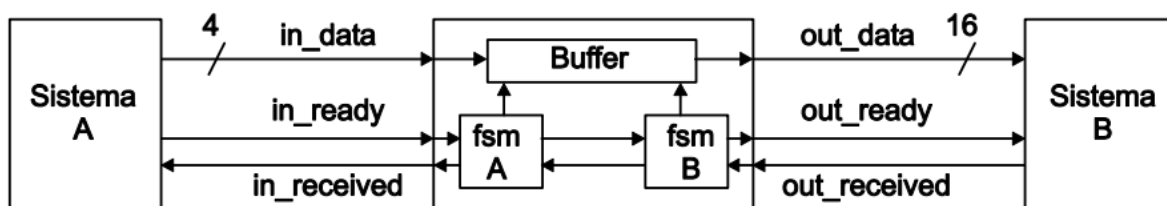


Nel testbench è stato verificato il corretto funzionamento delle memorie e del contatore nonché dell'uscita della rete combinatoria.

Handshake

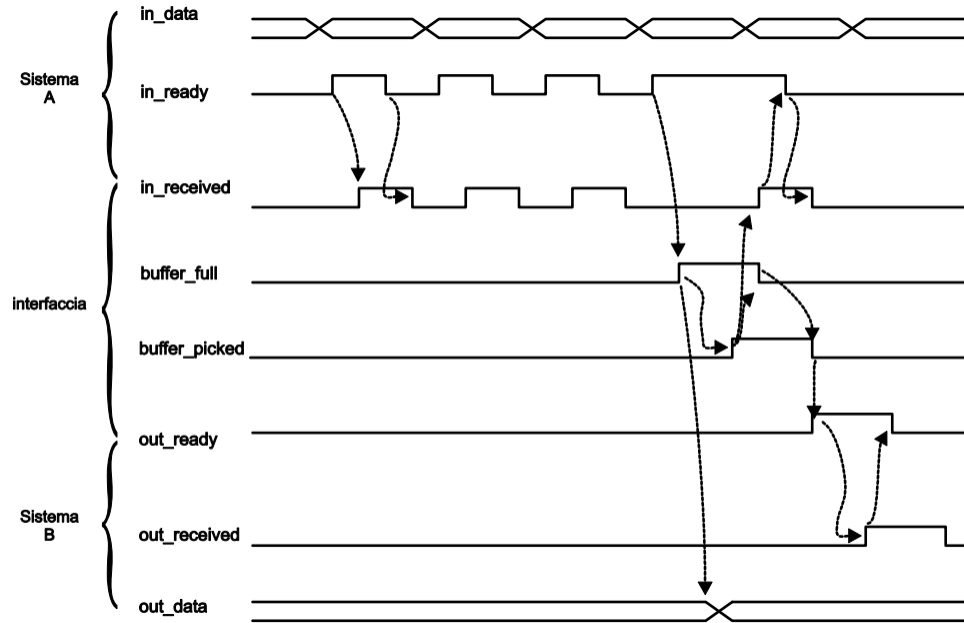
Esercizio 7 Progettare, implementare in VHDL e testare mediante simulazione un sistema composto da 2 nodi, A e B, che comunicano mediante un protocollo di handshaking. Il nodo A e il nodo B possiedono entrambi una memoria interna in cui sono memorizzate N stringhe di M bit, denominate $X(i)$ e $Y(i)$ rispettivamente ($i=0,..,N-1$). Il nodo A trasmette a B ciascuna stringa $X(i)$ utilizzando un protocollo di handshaking; B, ricevuta la stringa $X(i)$, calcola $S(i)=X(i)+Y(i)$ e immagazzina la somma in opportune locazioni della propria memoria interna. Per il progetto è possibile considerare una implementazione di tipo comportamentale per effettuare la somma, mentre è necessario prevedere esplicitamente un componente contatore sia nel sistema A sia nel sistema B per scandire la trasmissione/ricezione delle stringhe e per terminare la comunicazione.

7.1 Schematici

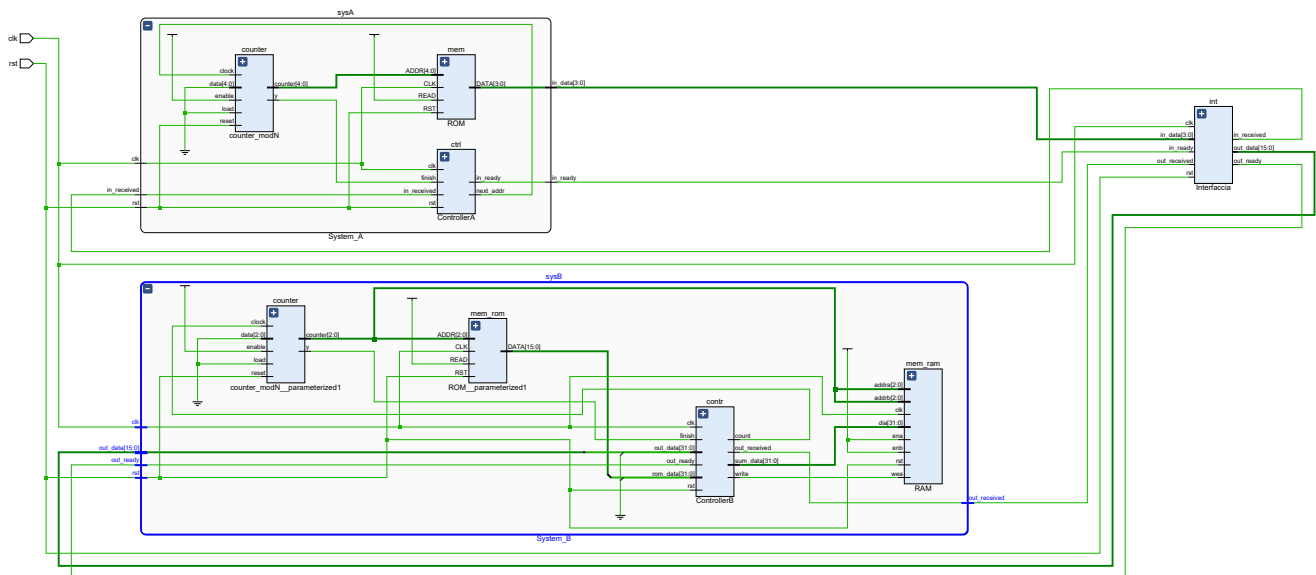


Si è fatta la scelta arbitraria che A possa inviare solo 4 bit per volta e che B ne riceva invece 16. Quindi è stata aggiunta un'interfaccia contenente un buffer che permetta di risolvere questa differenza di dimensione del parallelismo dei due sistemi.

I due sistemi comunicano con due automi a stati finiti che scandiscono i segnali di ready e received da e verso l'interfaccia. A loro volta le due FSM dialogano per coordinarsi sul prelievo del buffer con dei segnali di buffer_full e buffer_picked.



Nello schema è possibile vedere la tempificazione di questi segnali a livello teorico. Nella simulazione ci si attende di ottenere un risultato simile.



In VHDL i due sistemi sono stati implementati come da specifica usando dei componenti contatori. Sono presenti anche dei componenti controller che si occuperanno della gestione dei segnali verso l'interfaccia. Il sistema B come locazione interna di memoria utilizza una RAM.

Le ROM in questo esercizio leggono il contenuto da file e quindi vengono precaricate diversamente nei due sistemi. Questa scelta impedisce la sintesi dei componenti ma funziona senza problemi in simulazione a patto di avere i file immagine della memoria nella cartella di lavoro della simulazione.

7.2 Sistema A

Il sistema A è composto da una ROM contenente i valori da inviare a B, da un contatore che ne scandisce gli indirizzi e un segnale di *finish* e un controller di cui vediamo il codice.

7.2.1 Codice

Il controller di A riceve in ingresso i segnali di *clock*, e di *reset*, di *finish* e *in_received*, in uscita ha un segnale *next_addr* che viene usato come clock del contatore e *in_received* verso l'interfaccia.

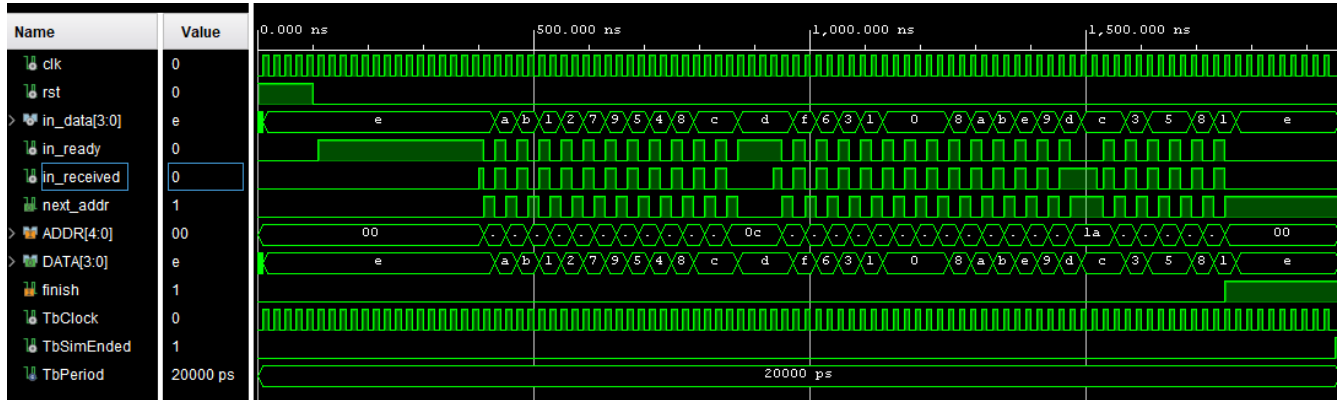
Fintanto che il trasferimento è in corso opera come segue.

- Se *in_received* è 0 allora l'interfaccia è in grado di ricevere un altro dato, in questo stato *next_addr* viene mantenuto a 0 e viene alzato *in_ready*.
- Se *in_received* è 1 avviene l'opposto, viene abbassato *in_ready* come ACK alla conferma di ricezione da parte dell'interfaccia e viene precaricato il prossimo valore della ROM facendo contare il contatore alzando *next_addr*.

```

1  entity ControllerA is
2      Port ( clk : in STD_LOGIC;
3            rst : in std_logic;
4            in_ready : out STD_LOGIC;
5            in_received : in STD_LOGIC;
6            next_addr : out STD_LOGIC;
7            finish : in STD_LOGIC);
8  end ControllerA;
9
10 architecture Behavioral of ControllerA is
11 begin
12
13 process(clk, rst)
14 begin
15     if rst = '1' then
16         next_addr <= '0';
17         in_ready <= '0';
18     elsif rising_edge(clk) then
19         if finish /= '1' then —Fine trasmissione
20             if in_received = '0' then
21                 — Se l'interfaccia ha ricevuto (e quindi pronta a ricevere un altro dato) alzo in_ready
22                 next_addr <= '0';
23                 in_ready <= '1';
24             elsif in_received = '1' then
25                 — Se l'interfaccia ha ricevuto abbasso in_ready e incremento il contatore
26                 in_ready <= '0';
27                 next_addr <= '1';
28             end if;
29         end if;
30     end if;
31 end process;
32
33 end Behavioral;
```

7.2.2 Simulazione



Nella simulazione `in_received` varia il suo valore a seconda del segnale `in_ready` in uscita dal sistema A. In particolare è stata sfruttata l'istruzione `wait until` per attendere la variazione di `in_ready` e commutare `in_received`. Occasionalmente sono stati introdotti dei ritardi nella risposta dell'interfaccia per verificare il corretto funzionamento anche in questo caso.

7.3 Sistema B

Il sistema B è del tutto simile al sistema A ma in questo caso possiede anche una RAM. Il suo controller ha in output anche un segnale per la gestione della somma dei valori sulla ROM locale con quelli ricevuti da A. Infatti sono state aggiunte due linee dati in ingresso da 16 bit `out_data` e `rom_data` e una in uscita verso la RAM da 32 bit `sum_data` oltre a un segnale di write.

7.3.1 Codice

```

1  entity ControllerB is
2      Port ( clk : in STD_LOGIC;
3            rst : in STD_LOGIC;
4            write : out std_logic;
5            count : out std_logic;
6            sum_data : out std_logic_vector(31 downto 0);
7            rom_data : in std_logic_vector(31 downto 0);
8            out_data : in std_logic_vector(31 downto 0);
9            out_ready : in std_logic;
10           out_received : out std_logic;
11           finish : in std_logic;
12       );
13 end ControllerB;
14
15 architecture Behavioral of ControllerB is
16     signal first : std_logic := '1';
17     begin
18     process (clk, rst)
19     begin
20         if rst = '1' then
21             first <= '1';
22             write <= '0';
23             count <= '0';
24             out_received <= '0';
25         elsif rising_edge(clk) then
26             if finish /= '1' then -- Fine ricezione
27                 if out_ready = '1' then -- Se l'interfaccia ha un output pronto lo preleva e alza out_received
28                     sum_data <= std_logic_vector(unsigned(rom_data) + unsigned(out_data));
29                     first <= '0';
30                     write <= '1';
31                     count <= '0';
32                     out_received <= '1';
33                 elsif out_ready = '0' then -- Se l'interfaccia abbassa out_ready chiaramente abbassiamo out_received e il
34                                         -- segnale di scrittura
35             end if;
36         end if;
37     end process;
38 end Behavioral;

```

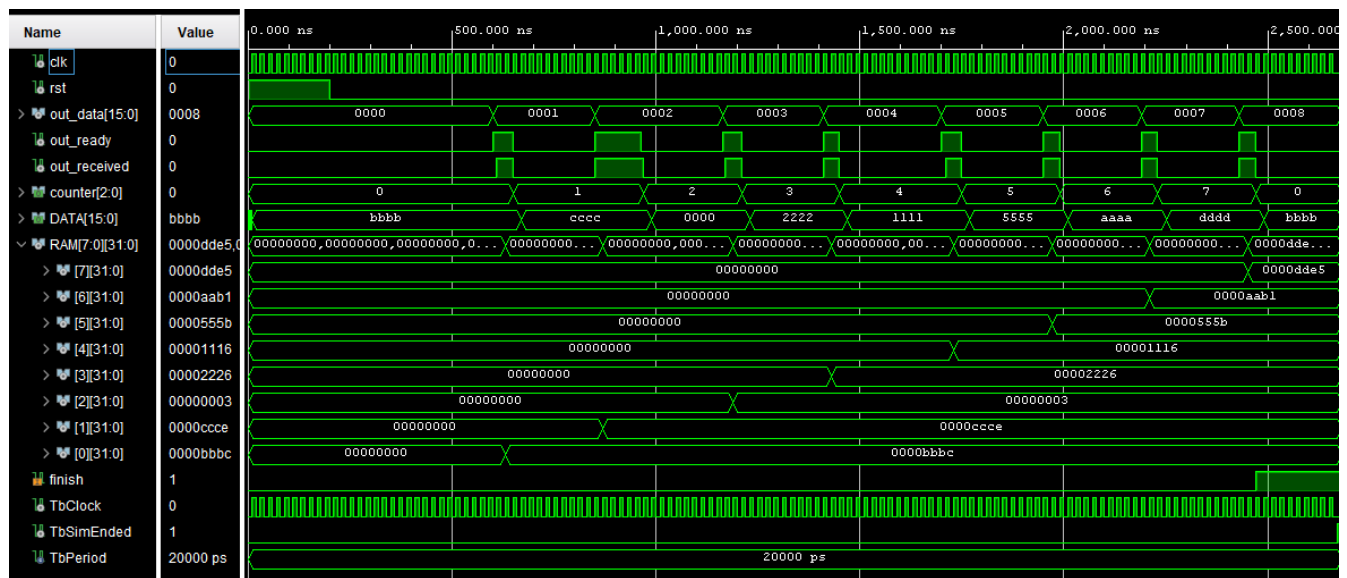
```

36     if first /= '1' then
37         count <= '1';
38     end if;
39     out_received <= '0';
40     write <= '0';
41 end if;
42 end if;
43 end if;
44 end process;
45
46 end Behavioral;

```

Eccetto la gestione dell'operazione di somma come menzionato precedentemente e una piccola modifica per la gestione del primo valore ricevuto (che non deve far aumentare il contatore) il codice del controller del sistema B è essenzialmente speculare a quello del sistema A.

7.3.2 Simulazione



La simulazione è analoga a quanto fatto per A. Anche in questo caso sono stati simulati dei leggeri ritardi e viene inoltre visualizzato lo stato della memoria RAM e gli operandi della somma.

7.4 Interfaccia

L'interfaccia è costituita da due automi a stati finiti che effettuano la comunicazione necessaria per la gestione di un buffer. Per renderli sintetizzabili si è scelto di dotare l'interfaccia di un clock e di realizzare una architettura sincrona. Vengono scambiati i messaggi di buffer full e buffer picked all'interno dell'interfaccia e i messaggi verso i due sistemi, in_ready e in_received verso A e out_ready e out_received verso B. Come abbiamo visto precedentemente l'interfaccia deve alzare il segnale in_received quando preleva il dato da A e abbassarlo quando poi A abbassa in_ready. Con B deve alzare out_ready quando out_received e quando quest'ultimo assume il valore logico 1 deve abbassare out_ready. Internamente vi è l'aggiunta della gestione del buffer: il segnale buffer_full viene reso alto dalla FSM A e viene abbassato quando viene ricevuta la risposta buffer_picked da FSM B.

7.4.1 Codice

```

1  entity Interfaccia is
2      Port ( clk: in std_logic;
3             rst: in std_logic;
4             in_data : in STD_LOGIC_VECTOR (3 downto 0);
5             out_data : out STD_LOGIC_VECTOR (15 downto 0);
6             in_ready : in STD_LOGIC;
7             in_received : out STD_LOGIC;
8             out_ready : out STD_LOGIC;
9             out_received : in STD_LOGIC);
10 end Interfaccia;
11
12 architecture Behavioral of Interfaccia is
13     type state_A is (waiting, receiving, received, full);
14     type state_B is (waiting, receiving, received, empty);
15     signal current_state_B: state_B := waiting;
16     signal current_state_A: state_A := waiting;
17     signal next_state_A: state_A := waiting;
18     signal next_state_B: state_B := waiting;
19     signal buffer_picked, buffer_full: std_logic := '0';
20     signal buf: std_logic_vector(15 downto 0) := X"0000";
21     signal count: integer range 0 to 3 := 0;
22 begin
23
24     state: process(clk, rst)
25     begin
26         if rst = '1' then
27             current_state_B <= waiting;
28             current_state_A <= waiting;
29         end if;
30         if rising_edge(clk) then
31             current_state_A <= next_state_A;
32             current_state_B <= next_state_B;
33         end if;
34     end process;
35
36     fsmA: process(in_ready, current_state_A, buffer_picked, rst)
37     begin
38         if rst = '1' then
39             in_received <= '0';
40             buffer_full <= '0';
41             next_state_A <= waiting;
42             count <= 0;
43         else
44             case current_state_A is
45                 when waiting =>
46                     in_received <= '0';
47                     if in_ready = '1' then
48                         next_state_A <= receiving;
49                     end if;
50                 when receiving =>
51                     case count is
52                         when 0 => buf(15 downto 12) <= in_data;
53                         count <= count+1;
54                         next_state_A <= received;
55                         when 1 => buf(11 downto 8) <= in_data;
56                         count <= count+1;
57                         next_state_A <= received;
58                         when 2 => buf(7 downto 4) <= in_data;
59                         count <= count+1;
60                         next_state_A <= received;
61                         when 3 => buf(3 downto 0) <= in_data;
62                         next_state_A <= full;
63                         buffer_full <= '1';
64                     end case;
65                     in_received <= '1';
66                 when received =>
67                     buffer_full <= '0';
68                     if in_ready = '0' then
69                         next_state_A <= waiting;
70                     end if;
71                 when full =>
72                     count <= 0;
73                     if buffer_picked = '1' then
74                         next_state_A <= received;
75                     end if;
76                 end case;
77             end if;
78         end process;
79
80     fsmB: process(buffer_full, out_received, current_state_B, rst)
81     begin
82         if rst = '1' then
83             out_ready <= '0';
84             buffer_picked <= '0';
85             next_state_B <= waiting;
86         else
87             case current_state_B is
88                 when waiting =>
89                     buffer_picked <= '0';

```

```

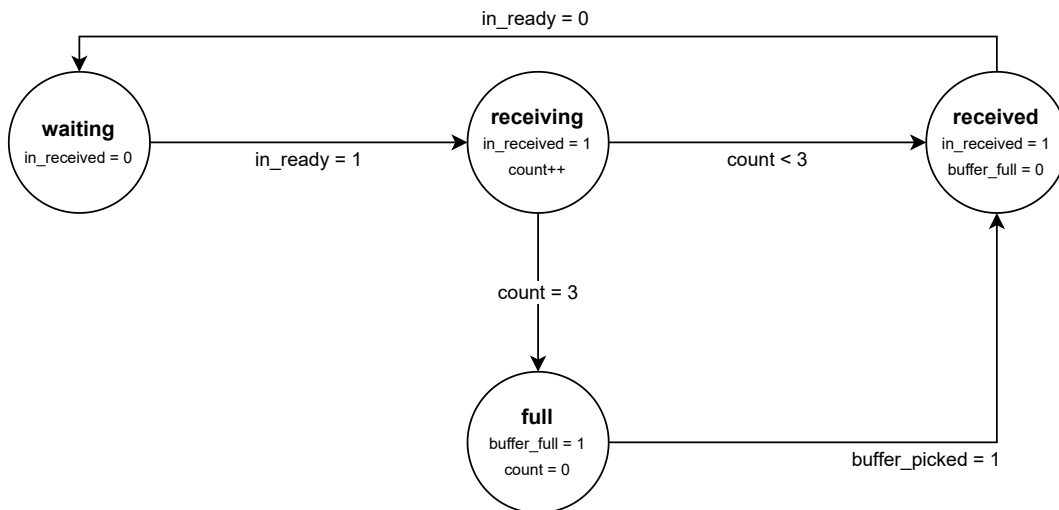
92     if buffer_full = '1' then
93         next_state_B <= receiving;
94     end if;
95     when receiving =>
96         out_data <= buf;
97         out_ready <= '1';
98         if out_received = '1' then
99             next_state_B <= received;
100        end if;
101    when received =>
102        buffer_picked <= '1';
103        out_ready <= '0';
104        if out_received = '0' then
105            next_state_B <= empty;
106        end if;
107    when empty =>
108        if buffer_full = '0' then
109            next_state_B <= waiting;
110        end if;
111    end case;
112 end if;
113 end process;
114 end Behavioral;

```

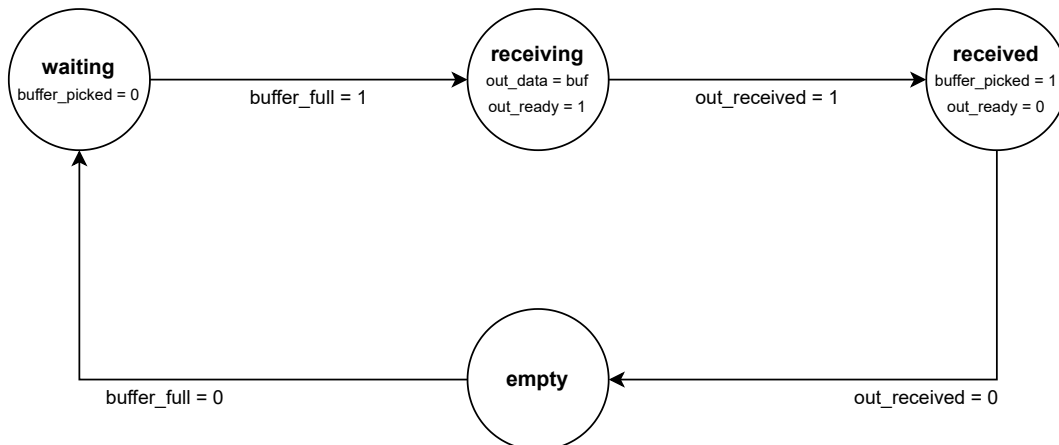
Per fare ciò vengono creati due process che implementano la FSM A e la FSM B. Le due macchine possiedono i seguenti stati:

- waiting, receiving, received, full per l'automa A. item waiting, receiving, received, empty per l'automa B.

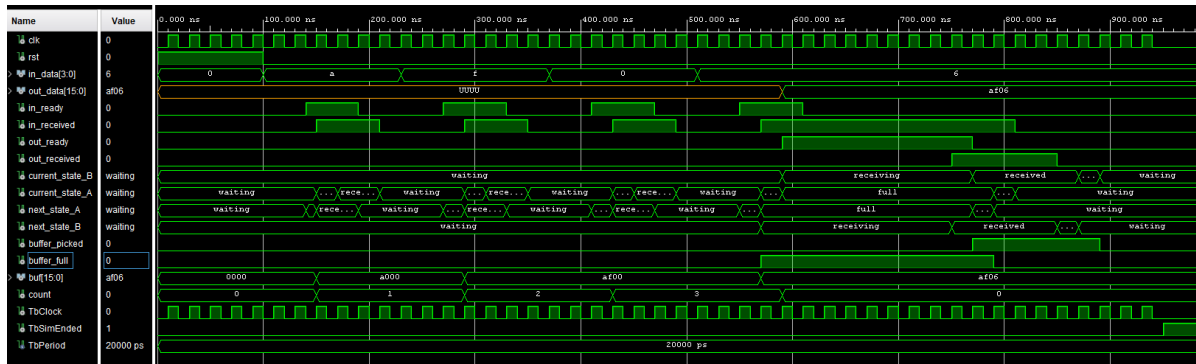
. Le transizioni tra questi stati vengono descritte dal seguente diagramma per l'automa A.



E dal seguente diagramma per l'automa B.

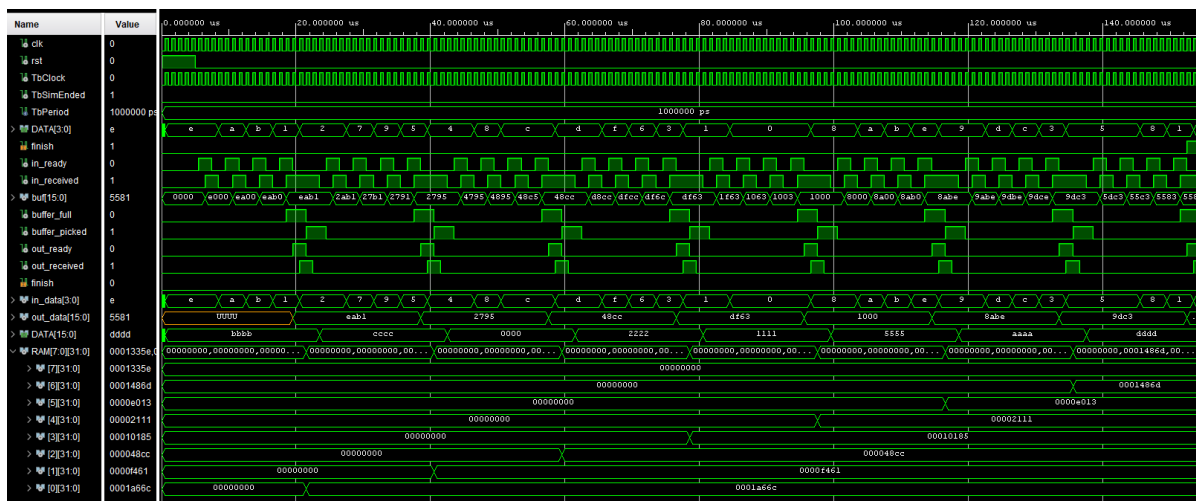


7.4.2 Simulazione



Quanto fatto è stato provato in simulazione fornendo all'interfaccia gli input simulati di uno scambio tipico da parte dei due sistemi.

7.5 Simulazione



Capitolo 8

Processore

A partire dall'implementazione fornita di un processore operante secondo il modello IJVM,

- 1. si proceda all'analisi dell'architettura mediante simulazione e si approfondisca lo studio del suo funzionamento per due istruzioni a scelta,*
- 2. si modifichi un codice operativo a scelta, documentando tutte le modifiche effettuate,*
- 3. (opzionale) si descriva il funzionamento del processore in merito alle istruzioni di input/output,*
- 4. (solo ove possibile) si sintetizzi il processore su FPGA.*

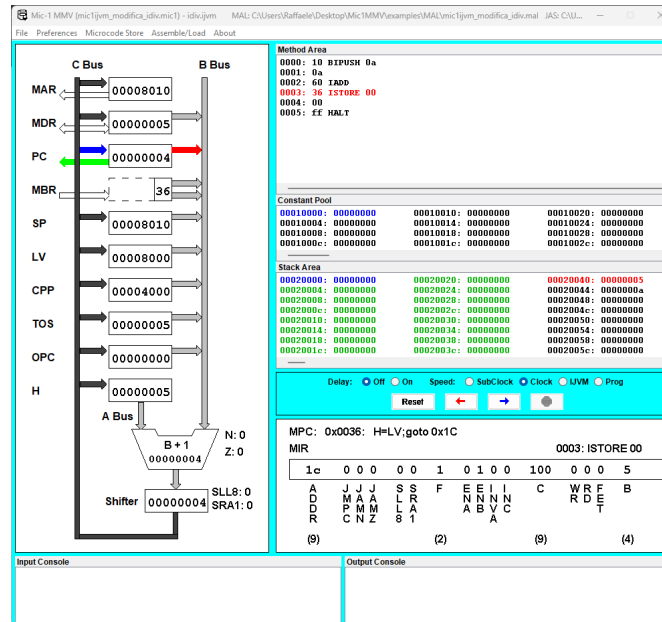
8.1 Analisi di ISTORE

La prima istruzione che si è scelto di analizzare è ISTORE. Per l'analisi è stato sfruttato il simulatore Mic-1 MMV¹ oltre alla classica simulazione con un testbench VHDL.

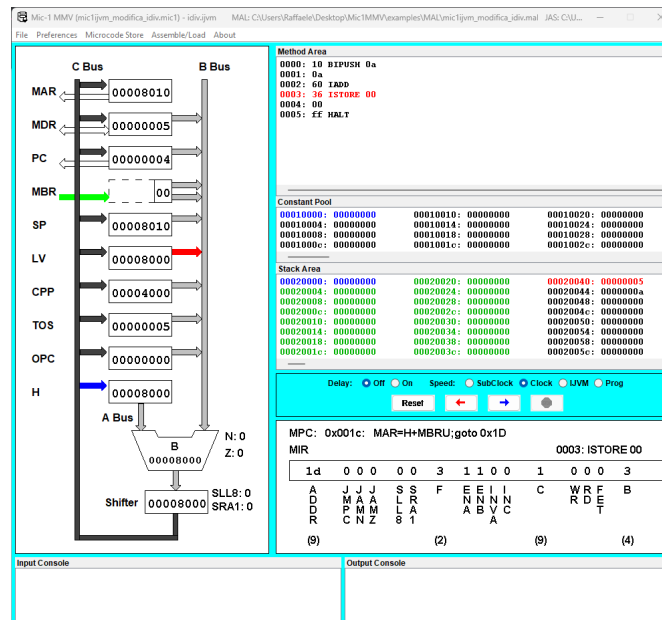
```
1 istore = 0x36:
2     H = LV
3     MAR = MBRU + H
4 istore_cont:
5     MDR = TOS; wr
6     SP = MAR = SP - 1; rd
7     PC = PC + 1; fetch
8     TOS = MDR; goto main
```

Il funzionamento di IADD è molto semplice. Innanzitutto MAR viene fatto puntare allo stack pointer decrementato di uno. Dopodichè si assegna al registro H la cima dello stack (TOS). Dopodichè si assegna a MDR e a TOS il valore dato dalla somma di MDR e H.

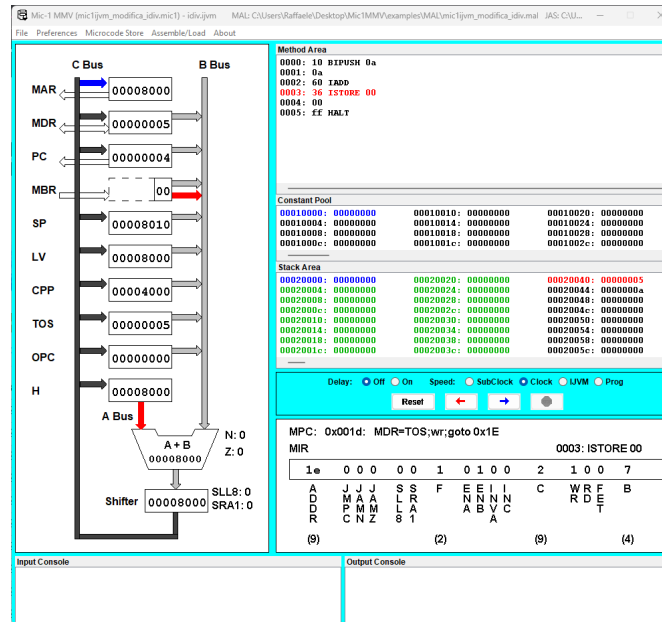
¹Mic-1 MMV. URL: <http://pdplab.it.uom.gr/teaching/tanenbaum/mic1mmv/mic1-Copying.html>.



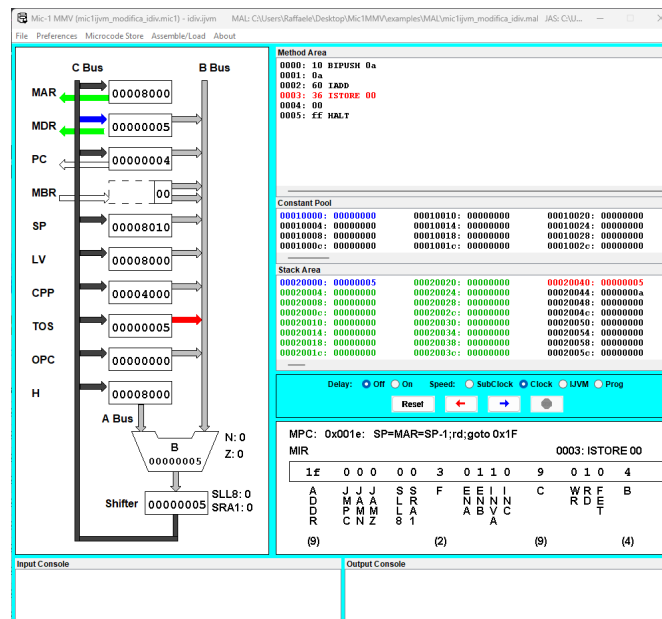
Per prima cosa viene assegnato al registro H l'indirizzo dello spazio di indirizzamento dedicato alle variabili locali (LV). Vediamo che nella parte relativa al bus C della microistruzione viene abilitata la scrittura di H.



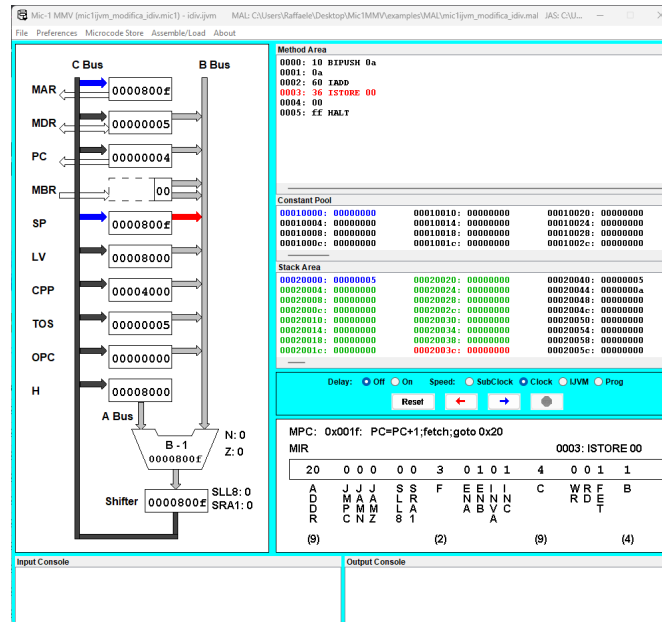
Il simulatore evidenzia con delle frecce le letture e le scritture avvenute sui registri relative all'istruzione che si è appena conclusa. Come secondo passo MAR viene eguagliato a MBR più H. In questo modo si realizza un accesso alle variabili locali secondo un meccanismo *base+spiazzamento*.



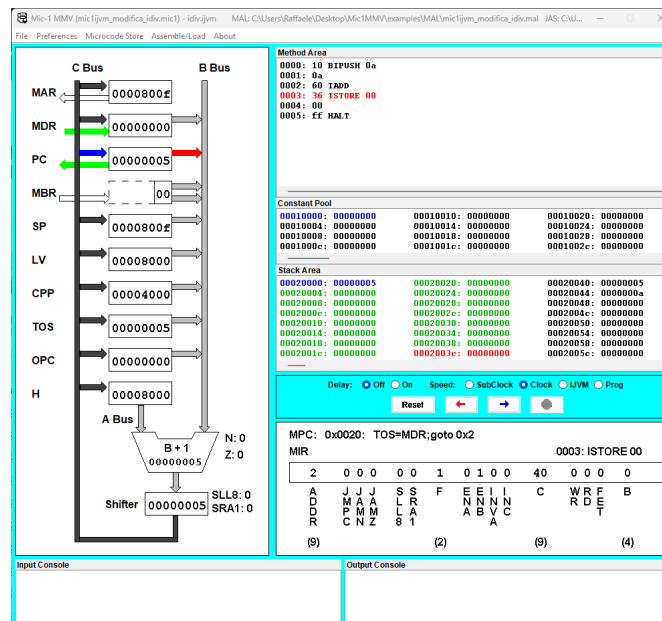
MDR che ora punta alla variabile locale viene eguagliato al valore TOS.



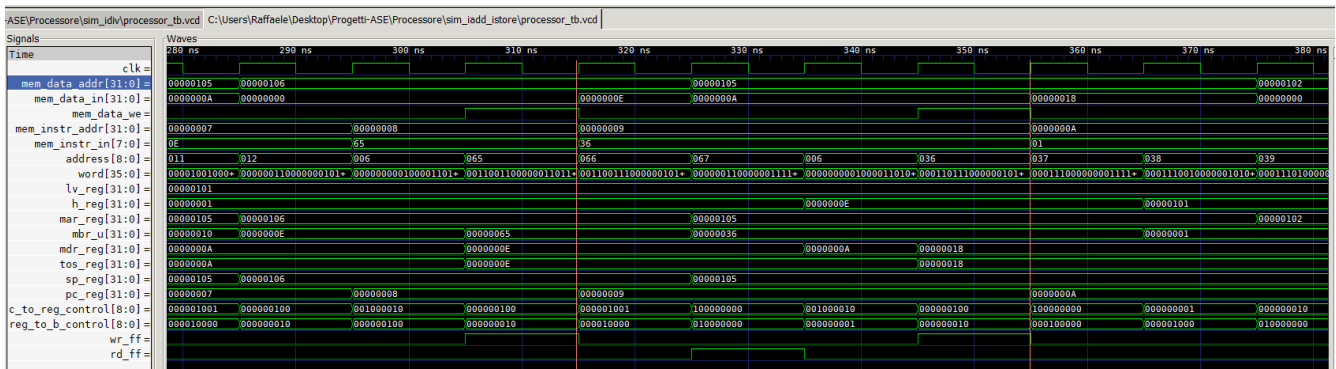
Lo stack pointer e MAR vengono impostati su SP-1.



Visto che la locazione di memoria successiva a quella di ISTORE contiene il l'offset del riferimento alla variabile locale (ricordiamo che MBR è stato aggiornato dall'ultimo fetch eseguito dal main e in questo modo assegna ad MBR l'offset della variabile locale), viene incrementato il PC e viene effettuato il fetch.



Come ultima istruzione viene aggiornato TOS.



Quanto descritto è stato anche provato mediante simulazione VHDL.

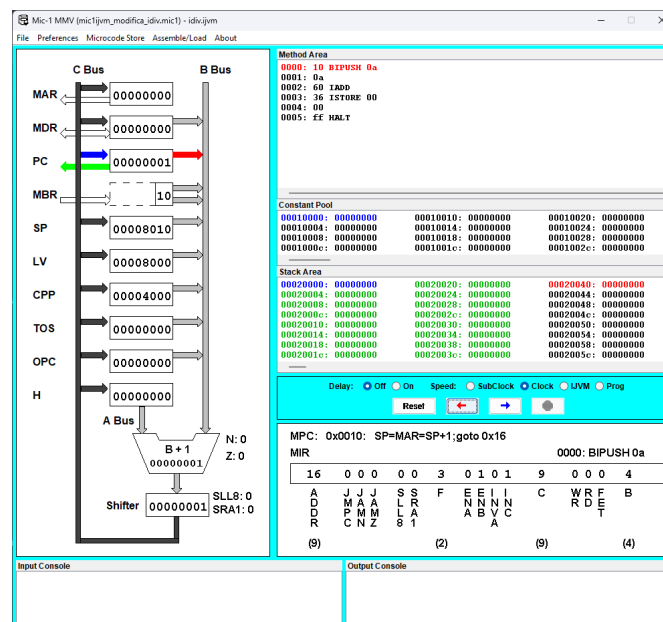
8.2 Analisi di BIPUSH

BIPUSH permette di posizionare un parametro in ingresso in cima allo stack.

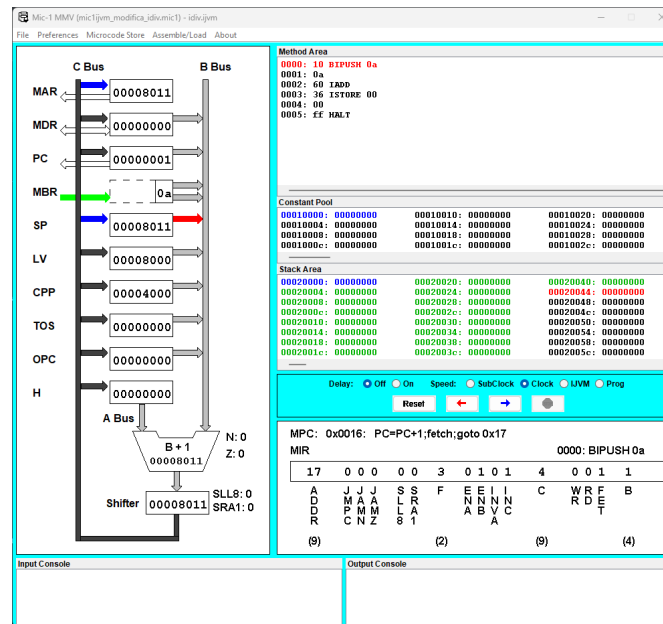
```

1 bipush = 0x10:
2     SP = MAR = SP + 1
3     PC = PC + 1; fetch
4     MDR = TOS = MBR; wr; goto main

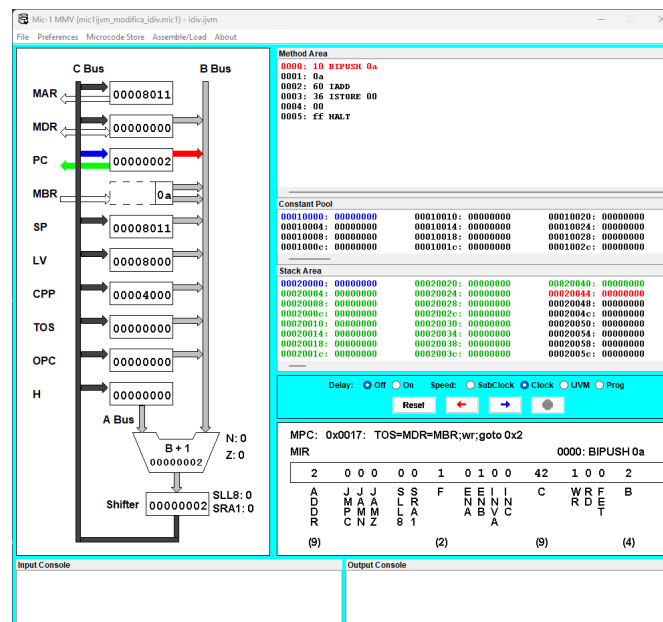
```



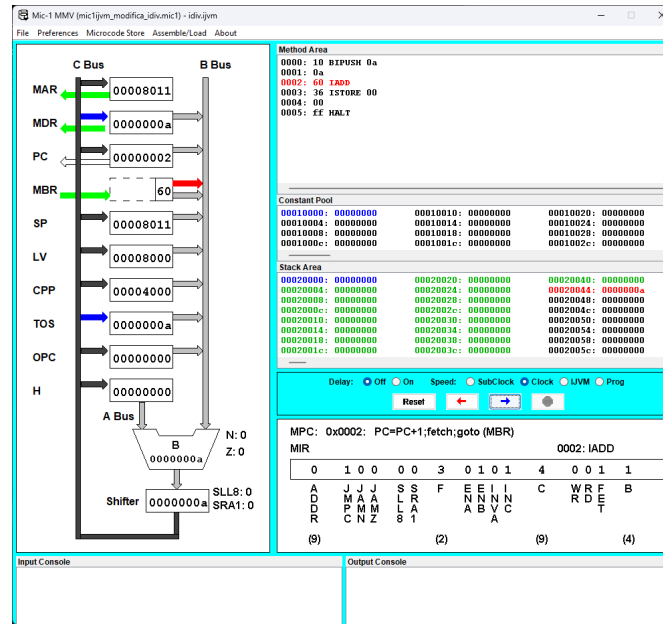
Il primo passo è quello di incrementare lo stack pointer di uno, il risultato viene assegnato a SP e a MAR, in questo modo abbiamo "allocato" sullo stack uno spazio per un valore.



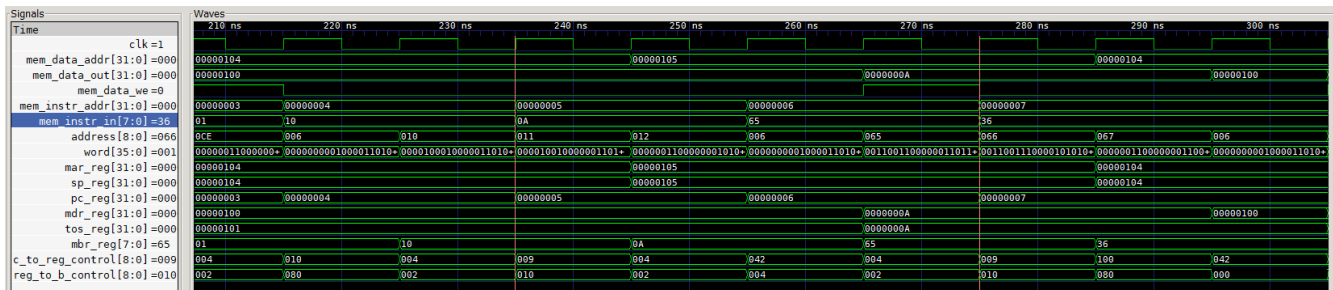
Viene incrementato il PC e viene eseguita l'istruzione di fetch. Ciò viene fatto perchè il parametro passato alla BIPUSH si trova nella locazione di memoria immediatamente successiva all'istruzione stessa, facendo il fetch portiamo il valore di tale locazione (e quindi il dato che vogliamo mettere in cima allo stack) nel registro MBR.



Si assegnano a MDR e a TOS il valore contenuto in MBR.



Come risultato finale vediamo che all'inizio dell'istruzione successiva il valore dato alla BIPUSH (0xA) si trova adesso in cima allo stack.



Il dettaglio di quanto fatto e i vari segnali di controllo dei bus B e C sono stati analizzati anche mediante un testbench in cui è inoltre vedere anche il dettaglio degli indirizzi delle istruzioni del control store.

8.3 Modifica di IADD

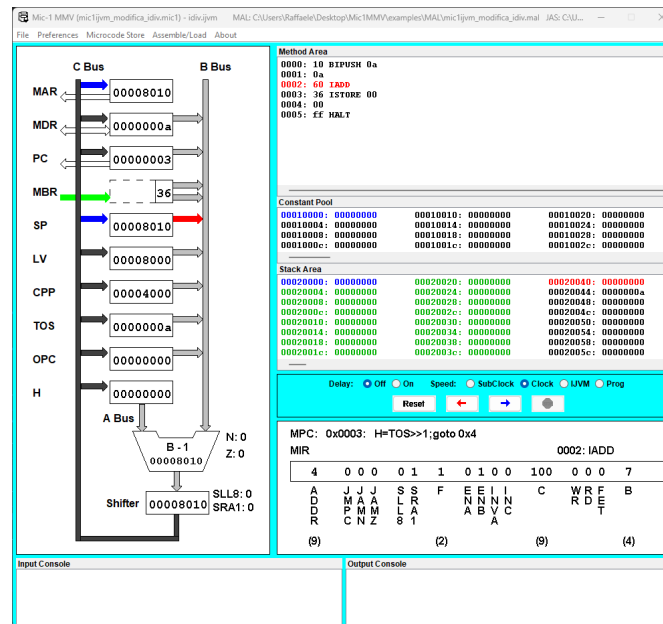
Si è deciso di modificare l'istruzione IADD affinché sfruttando lo shift register effettuasse una divisione per 2.

```

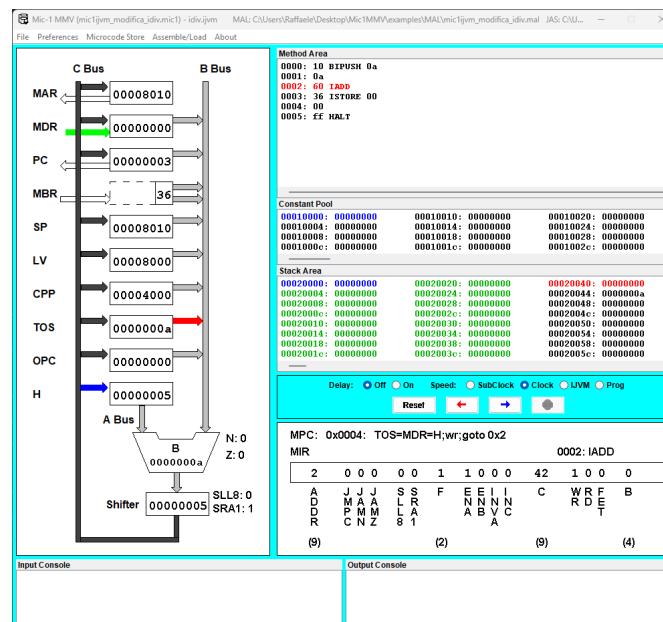
1 iadd = 0x65:
2     MAR = SP = SP - 1; rd
3     H = TOS >> 1
4     MDR = TOS = H; wr; goto main

```

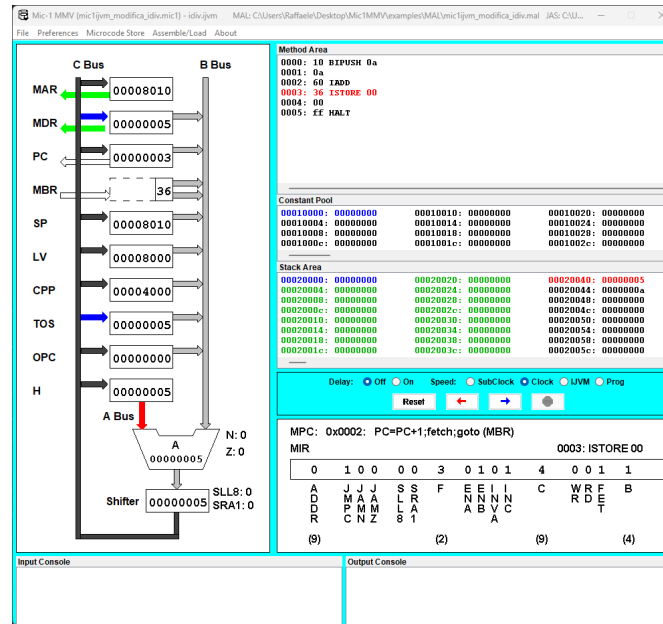
A livello di linguaggio MAL facciamo lo shift a destra di TOS e assegnamo il risultato a H, tale risultato viene poi assegnato a TOS e a MDR.



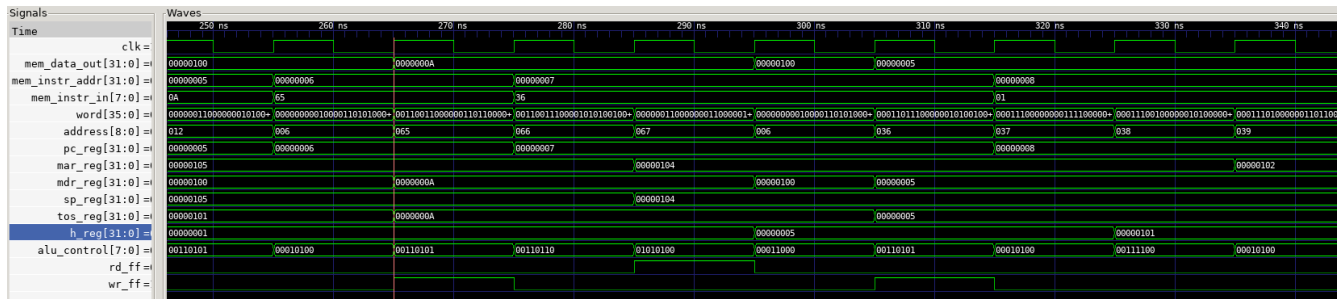
La prima istruzione decrementa SP e lo assegna a MAR. La seconda assegna ad H il registro TOS shiftato di 1. Vediamo che il segnale SRA1 in ingresso allo shifter a valle dell'adder è alto e per quanto riguarda il controllo del bus B questo ha come segnale 0x7 che indica la lettura dal registro TOS.



La seconda microistruzione assegna a MDR e a TOS il valore di H e termina l'istruzione con un goto al main. Il valore 0x42 per il controllo del bus C indica la scrittura dei registri TOS e MDR usando il valore di uscita dello shifter.



Come test è stato dato in ingresso 0xA, e come previsto nell'area stack il risultato è 0x5 (10 diviso 2). Quanto fatto è stato provato anche con un simulatore VHDL, in cui si può meglio vedere il dettaglio dell'ordine di esecuzione delle microistruzione e la tempificazione in relazione al clock.



8.4 Istruzioni di I/O

Il MIC-1 supporta di base l'I/O memory-mapped ossia è prevista l'associazione tra i registri della periferica (spesso usando un modello Dato-Controllo-Stato) e delle locazioni di memoria del dispositivo. Tale associazione, può essere realizzata mediante dei multiplexer che fungono da "proxy" tra il processore e la periferica. Quando il processore genera un indirizzo questo viene valutato (dal multiplexer o da altri meccanismi più raffinati) e se tale indirizzo appartiene allo spazio riservato alle periferiche il processore leggerà (o scriverà) il dato dalla periferica invece che dalla memoria. Per quanto riguarda le istruzioni utilizzate per l'I/O è possibile implementare due istruzioni di questo tipo.

```

1 gstore = 0xD0:
2   PC = PC + 1; fetch
3   H = MBRU << 8
4   H = MBRU OR H
5   MAR = H + CPP; rd
6   empty

```

```

7      MAR = MDR;
8      MDR = TOS; wr
9      SP = MAR = SP - 1; rd
10     PC = PC + 1; fetch
11     TOS = MDR; goto main
12
13 gload = 0xDC:
14     PC = PC + 1; fetch
15     H = MBRU << 8
16     H = MBRU OR H
17     MAR = H + CPP; rd
18     empty
19     MAR = MDR; rd;
20     MAR = SP = SP + 1
21     PC = PC + 1; fetch; wr
22     TOS = MDR; goto main

```

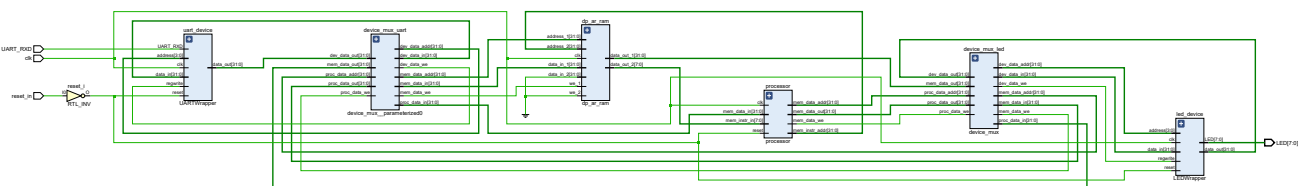
Entrambe GLOAD e GSTORE prelevano dal registro MBR (che grazie all'incremento di PC e al fetch corrisponde al valore passato all'istruzione) l'indirizzo di una variabile che contiene a sua volta l'indirizzo all'area di memoria memory mapped e assegnano tale indirizzo a MAR.

- GSTORE assegna a MDR (ossia al registro della periferica) il valore di TOS e fa una write all'indirizzo puntato da MAR, viene decrementato SP e viene assegnato a MAR e vengono aggiornati PC, MDR e TOS.
- GLOAD invece fa subito la read del valore contenuto nel nuovo indirizzo puntato da MAR, incrementa SP e lo assegna a MAR, in modo tale da scrivere in TOS il valore di MDR (che sarebbe il dato proveniente dalla periferica), nel mentre viene incrementato il PC e viene aggiornato TOS.

8.5 Sintesi su board FPGA

Sulla board FPGA si è deciso di sintetizzare il MIC-1 collegandolo a una periferica UART in ascolto dalla porta USB e a una periferica LED per la visualizzazione dei messaggi ricevuti. In pratica viene ricreato l'esercizio 9 ma con l'ausilio di un processore che gestirà in automatico la ricezione dei caratteri.

8.5.1 Schematici



Le periferiche UART e LED sono state connesse mediante due mux in cascata. Il primo controlla il decimo bit dell'indirizzo in uscita dal processore, se è 1 allora dati in uscita, l'indirizzo e il

segnale di scrittura vengono passati alla periferica LED. In caso contrario i valori della CPU e della memoria vengono passati al secondo mux che verifica invece il nono bit e permette alla CPU l'accesso alla memoria se questo è basso, altrimenti viene fatto l'accesso ai registri della periferica.

8.5.2 Codice VHDL

Le due periferiche sono state inserite all'interno di un wrapper che consente l'interfacciamento mediante registri. A titolo di esempio viene mostrato il wrapper della periferica UART.

```

1  entity UARTWrapper is
2  port (
3      clk      : in  std_logic;           --! Clock
4      reset    : in  std_logic;           --! Reset sincrono attivo alto
5      regwrite  : in  std_logic;           --! Enable scrittura registri
6      address   : in  std_logic_vector(31 downto 0); --! Indirizzo registro
7      data_in   : in  std_logic_vector(31 downto 0); --! Dati
8      data_out  : out std_logic_vector(31 downto 0); --! Datiend UARTWrapper;
9      UART_RXD : in  std_logic
10 );
11 end UARTWrapper;
12
13 architecture Structural of UARTWrapper is
14 component UARTcomponent
15     Generic (
16         BAUD_DIVIDE_G : integer := 14; --115200 baud
17         BAUD_RATE_G   : integer := 231
18     );
19     Port (
20         TXD : out std_logic := '1';
21         RXD : in  std_logic;
22         CLK : in  std_logic; --Master Clock
23         DBIN : in  std_logic_vector (7 downto 0); --Data Bus in
24         DBOUT : out std_logic_vector (7 downto 0); --Data Bus out
25         RDA : inout std_logic; --Read Data Available( alto quando il dato e' disponibile nel registro
26         rdReg)
27         TBE : inout std_logic := '1'; --Transfer Bus Empty(se 0 significa che il bus e' occupato per una
28         trasmissionione)
29         RD : in  std_logic; --Read Strobe(se 1 significa "leggi")
30         WR : in  std_logic; --Write Strobe(se 1 significa "scrivi")
31         PE : out std_logic; --Parity Error Flag
32         FE : out std_logic; --Frame Error Flag
33         OE : out std_logic; --Overwrite Error Flag
34         RST : in  std_logic := '0'; --Master Reset
35     end component;
36     type regbank is array (3 downto 0) of std_logic_vector(31 downto 0);
37
38     -- Registri
39     signal control_regs : regbank;
40     signal address_reg : std_logic_vector(31 downto 0) := "0000";
41     signal temp_rda, temp_rd : std_logic := '0';
42     signal temp_dbout : std_logic_vector(7 downto 0) := x"00";
43
44     alias rda : std_logic_vector(31 downto 0)
45     is control_regs(0);
46     alias rd : std_logic_vector(31 downto 0)
47     is control_regs(1);
48     alias dbout : std_logic_vector(31 downto 0)
49     is control_regs(2);
50
51     attribute ram_style : string;
52     attribute ram_style of control_regs : signal is "registers";
53
54 begin
55     UART_DEVICE: UARTcomponent generic map(BAUD_RATE_G => 868,BAUD_DIVIDE_G => 54 )
56     port map(
57         clk => clk ,
58         rst => reset ,
59         TXD => open ,
60         RXD => UART_RXD,
61         WR => '0' ,
62         dbin => (others => '0') ,
63         rd => temp_rd ,
64         RDA => temp_rda ,
65         FE => open ,
66         OE => open ,
67         PE => open ,
68         dbout => temp_dbout
69     );
70
71     regs : process (clk) is
72     begin
73         if clk'event and clk = '1' then
74             -- Reset
75             if (reset = '1') then
76                 control_regs(0) <= (others => '0');
77                 control_regs(1) <= (others => '0');
78                 control_regs(2) <= (others => '0');

```

```

77     end if;
78     -- Scrittura registri
79     if regwrite = '1' and to_integer(unsigned(address(3 downto 2))) = 1 then
80         control_regs(to_integer(unsigned(address(3 downto 2)))) <= data_in;
81     end if;
82     dbout(7 downto 0) <= temp_dbout;
83     temp_rd <= rd(0);
84     rda(0) <= temp_rda;
85 end if;
86 end process;
87
88
89
90 data_out <= control_regs(to_integer(unsigned(address(3 downto 2))));
91 end Structural;

```

8.5.3 Codice IJVM

Il programma IJVM implementa il meccanismo del polling tra il processore e la periferica UART.

```

1  .constant
2  rda_reg 0x200
3  rd_reg 0x204
4  data_reg 0x208
5  led_reg 0x400
6
7  .endconstant
8
9  .main
10
11 label1: GLOAD rda_reg
12         IFEQ label1
13         GLOAD data_reg
14         GSTORE led_reg
15         BIPUSH 1
16         GSTORE rd_reg
17 label2: GLOAD rda_reg
18         BIPUSH 1
19         IF_ICMPEQ label2
20         BIPUSH 0x0
21         GSTORE rd_reg
22         GOTO label1
23         HALT
24 .endmethod

```

Sono presenti 4 costanti che sono gli indirizzi dei registri delle periferiche. La periferica UART è sempre in ascolto sulla porta USB. Il processore in loop preleva il valore del registro `rda_reg` per verificare la ricezione di un nuovo carattere. Quando il contenuto di `rda_reg` è 1, viene eseguita `GLOAD` sul registro `data_reg` che contiene il carattere ricevuto dalla UART (segnale `DBOUT`). Il valore così ottenuto viene visualizzato (`GSTORE`) sui LED. A questo punto viene impostato a 1 `rd_reg` e si attende che `rda_reg` valga 0. Quando ciò accade viene posto di nuovo a 0 `rd_reg` e il ciclo si ripete.

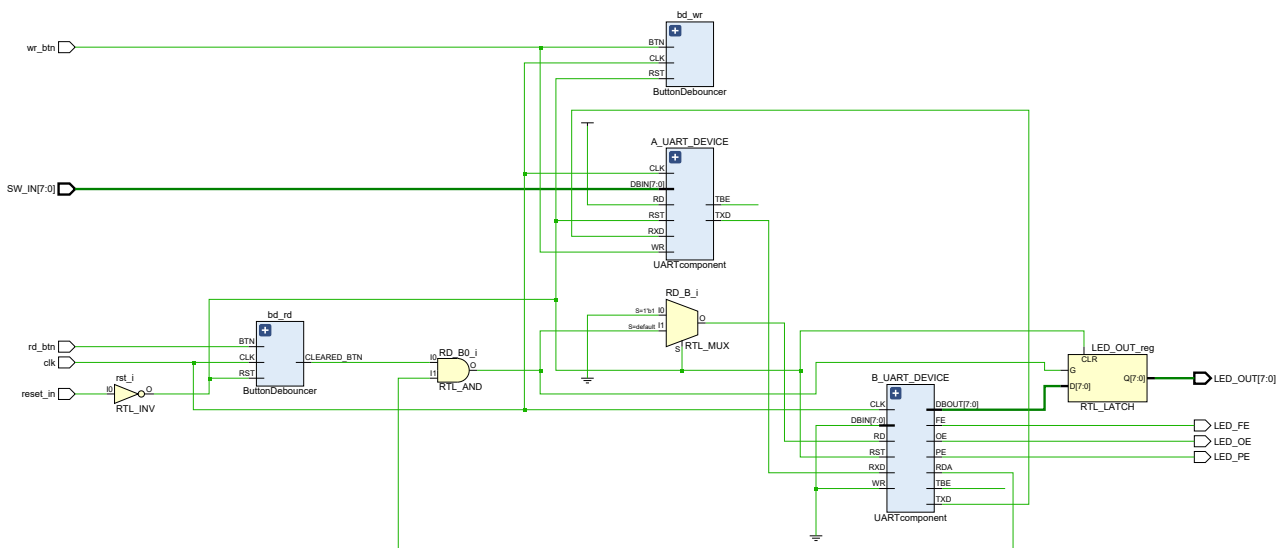
Capitolo 9

Seriale

9.1 Esercizio 9.1

Sfruttando l'implementazione fornita dalla Digilent di un dispositivo UART (componente *RS232RefComp.vhd*), progettare e implementare in VHDL un sistema costituito da 2 nodi A e B collegati tra loro mediante una interfaccia seriale. Il sistema A acquisisce una stringa di 8 bit dall'utente (mediante gli switch della board di sviluppo) e la invia mediante la seriale al sistema B, che la manda in output sui led della board di sviluppo.

9.1.1 Schematici



Sono stati creati due dispositivi UART, uno per nodo, opportunamente collegati. Abbiamo inoltre due button debouncer che sono connessi a due pulsanti che segnalano rispettivamente la scrittura per A e la lettura per B.

9.1.2 Codice

```

1  entity Seriale is
2      Port ( clk : in STD_LOGIC;
3             reset_in : in STD_LOGIC;
4             SW_IN : in STD_LOGIC_VECTOR (7 downto 0);
5             LED_OUT : out STD_LOGIC_VECTOR (7 downto 0);
6             LED_PE : out std_logic;
7             LED_FE : out std_logic;
8             LED_OE : out std_logic;
9             rd_btn : in std_logic;
10            wr_btn : in STD_LOGIC);
11 end Seriale;
12
13 architecture Structural of Seriale is
14
15     component UARTcomponent
16         Generic (
17             BAUD_DIVIDE_G : integer := 14;  --115200 baud
18             BAUD_RATE_G : integer := 231
19         );
20         Port (
21             TXD : out std_logic := '1';
22             RXD : in std_logic;
23             CLK : in std_logic;           --Master Clock
24             DBIN : in std_logic_vector (7 downto 0); --Data Bus in
25             DBOUT : out std_logic_vector (7 downto 0); --Data Bus out
26             RDA : inout std_logic;         --Read Data Available( alto quando il dato    disponibile nel registro
27                                     rdReg)
28             TBE : inout std_logic := '1';    --Transfer Bus Empty(se 0 significa che il bus    occupato per una
29                                     trasmissione)
30             RD : in std_logic;               --Read Strobe(se 1 significa "leggi")
31             WR : in std_logic;               --Write Strobe(se 1 significa "scrivi")
32             PE : out std_logic;              --Parity Error Flag
33             FE : out std_logic;              --Frame Error Flag
34             OE : out std_logic;              --Overwrite Error Flag
35             RST : in std_logic := '0');      --Master Reset
36     end component;
37
38     component ButtonDebouncer
39         generic (
40             CLK_period: integer := 10;  -- periodo del clock in nanosec
41             btn_noise.time: integer := 6500000 --durata dell'oscillazione in nanosec
42         );
43         Port ( RST : in STD_LOGIC;
44               CLK : in STD_LOGIC;
45               BTN : in STD_LOGIC;
46               CLEARED_BTN : out STD_LOGIC);
47     end component;
48
49     signal rst, wr_btn_clr, rd_btn_clr, A2B, B2A, RDA_B: std_logic := '0';
50     signal RD_B: std_logic := '0';
51     signal data_rx: std_logic_vector(7 downto 0);
52 begin
53     rst <= not reset_in;
54     A_UART_DEVICE: UARTcomponent generic map(BAUD_RATE_G => 868, BAUD_DIVIDE_G => 54 )
55     port map(
56         clk => clk,
57         rst => rst,
58         TXD => A2B,
59         RXD => B2A,
60         WR => wr_btn,
61         dbin => SW_IN,
62         rd => '1'
63     );
64
65     B_UART_DEVICE: UARTcomponent generic map(BAUD_RATE_G => 868, BAUD_DIVIDE_G => 54 )
66     port map(
67         clk => clk,
68         rst => rst,
69         TXD => B2A,
70         RXD => A2B,
71         WR => '0',
72         dbin => (others => '0'),
73         rd => RD_B,
74         RDA => RDA_B,
75         FE => LED_FE,
76         OE => LED_OE,
77         PE => LED_PE,
78         dbout => data_rx
79     );
80
81
82     bd_wr: ButtonDebouncer generic map(CLK_period => 10, btn_noise.time => 6500000)
83     port map(
84         rst => rst,
85         clk => clk,
86         btn => wr_btn,
87         cleared_btn => wr_btn_clr
88     );
89
90     bd_rd: ButtonDebouncer generic map(CLK_period => 10, btn_noise.time => 6500000)
91     port map(

```

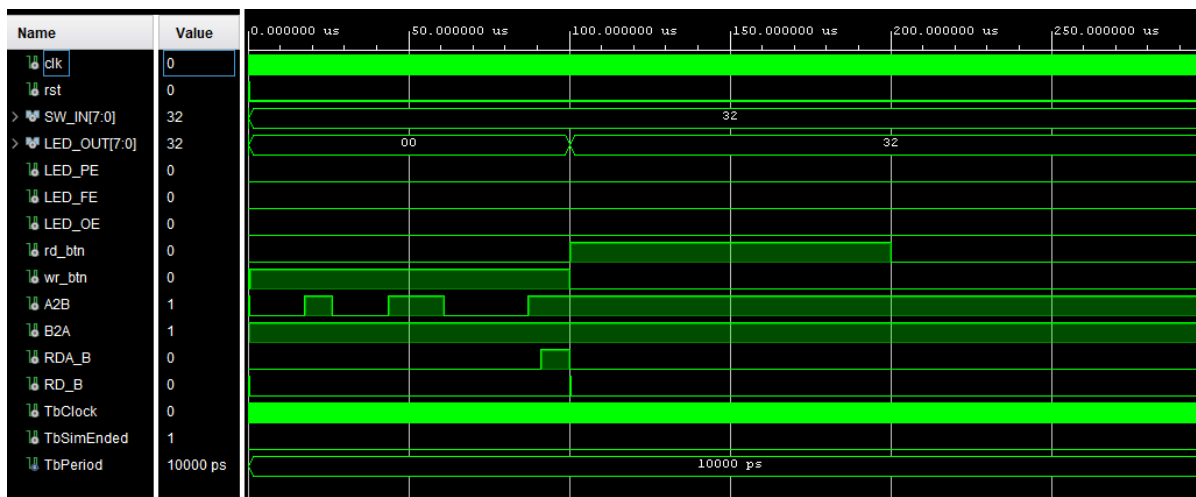
```

92     rst => rst ,
93     clk => clk ,
94     btn => rd_btn ,
95     cleared_btn => rd_btn_clr
96 );
97
98
99 LED_OUT <= (others => '0') when rst = '1' else
100     data_rx when rd_btn_clr = '1' and RDA_B = '1';
101 RD_B <= '0' when rst = '1' else
102     '1' when rd_btn_clr = '1' and RDA_B = '1' else
103     '0';
104 LED_RDA <= RDA_B;
105
106 end Structural;

```

Il codice riflette i collegamenti fatti a livello strutturale nello schematico.

9.1.3 Simulazione



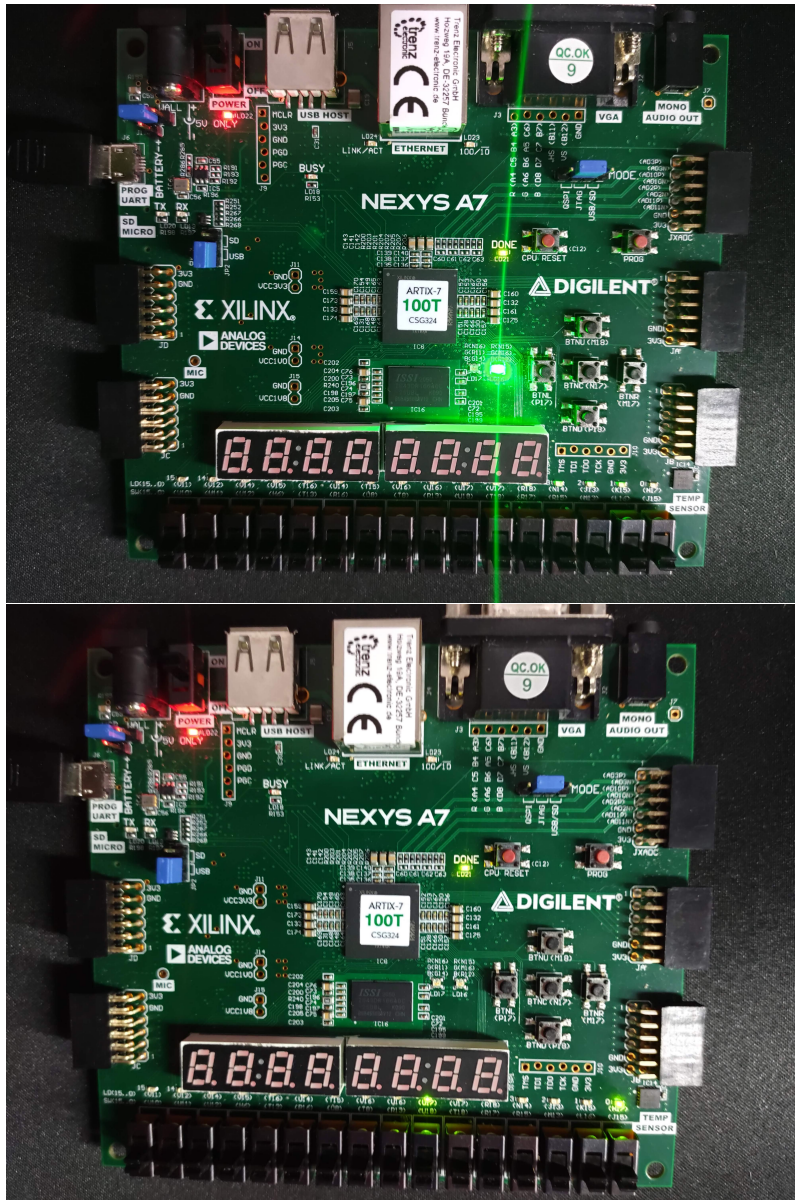
Possiamo vedere le variazioni del segnale A2B scatenate dall'avvio della trasmissione da parte dell'UART. Quando la trasmissione è completata il segnale RDA di B è alto ed è possibile dare il comando di lettura dall'UART. Se vi sono errori i segnali relativi agli errori di parità, framing o overrun assumeranno il valore logico 1.

9.1.4 Sintesi su board FPGA

Sulla board FPGA vengono usati gli switch per l'ingresso dei valori da trasmettere. Un pulsante scatena la trasmissione di A mentre un secondo pulsante permette la lettura del dato se RDA (collegato anch'esso a un LED) è alto. I valori vengono mostrati sull'array di LED della board.

9.2 Esercizio 9.2b

UART_PC: il sistema realizza la comunicazione fra un nodo A rappresentato da un componente sintetizzato su un FPGA e un nodo B rappresentato da un terminale seriale in esecuzione su PC (es. Termite), previa connessione di PC e board tramite dispositivo fisico RS232 (uno degli endpoint di comunicazione è rappresentato dal PC). Il componente A acquisisce una stringa di 8 bit che rappresenta un carattere in codifica ASCII fornita dall'utente mediante gli switch della board di sviluppo, e la invia mediante il dispositivo UART al terminale B in esecuzione sul PC, in



Capitolo 10

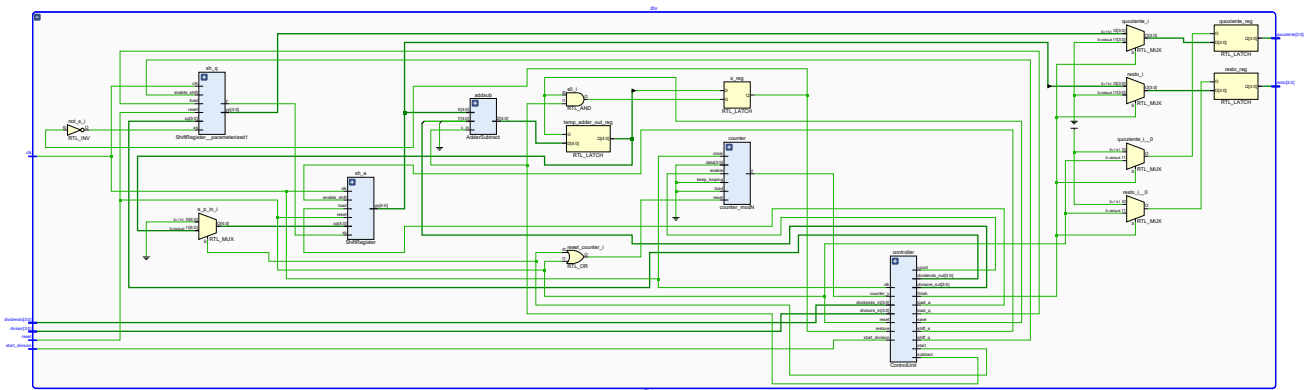
Divisore

Progettare ed implementare in VHDL una macchina aritmetica sequenziale a scelta fra le seguenti:

- ...
- *divisore restoring*, per effettuare la divisione intera fra due stringhe A e B di 4 bit ciascuna.

Opzionalmente, la macchina implementata può essere sintetizzata su FPGA e testata mediante l'utilizzo dei dispositivi di input/output (switch, bottoni, led, display) presenti sulla board di sviluppo in dotazione al gruppo. La modalità di utilizzo degli stessi è a completa discrezione degli studenti.

10.1 Schematici



All'interno del divisore sono presenti due shift register Q (quoziente) e A (resto) un adder/subtractor a 4 bit, un contatore e una unità di controllo. L'uscita seriale di A è collegata all'ingresso di Q. Inoltre A riceve come ingresso parallelo i risultati dell'adder. Le operazioni di shift e di load dei due registri sono pilotate dalla control unit.

10.2 Codice

Per questo progetto si è scelto di utilizzare un controller microprogrammato. I valori memorizzati sulla ROM implementano i vari segnali di controllo del divisore.

```

1  library ieee;
2  use ieee.STD_LOGIC_1164.all;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity MicroROM is
6  Port (
7      pc: in unsigned(3 downto 0);
8      pc_next: out unsigned(3 downto 0);
9      shift_q: out std_logic;
10     shift_a: out std_logic;
11     load_q: out std_logic;
12     load_a: out std_logic;
13     start: out std_logic;
14     subtract: out std_logic;
15     save: out std_logic;
16     count: out std_logic;
17     finish: out std_logic
18 );
19 end MicroROM;
20
21 architecture synth of MicroROM is
22 type Control_type is record
23     pc_next: unsigned(3 downto 0);
24     shift_q: std_logic;
25     shift_a: std_logic;
26     load_q: std_logic;
27     load_a: std_logic;
28     start: std_logic;
29     subtract: std_logic;
30     save: std_logic;
31     count: std_logic;
32     finish: std_logic;
33 end record;
34
35 constant idle: Control_type := ("0000", '0', '0', '0', '0', '0', '1', '0', '0', '0'); --0000
36 constant init: Control_type := ("0010", '0', '0', '1', '1', '1', '1', '0', '0', '0'); --0001
37 constant shiftA: Control_type := ("0011", '0', '1', '0', '0', '0', '1', '0', '0', '0'); --0010
38 constant save_sub: Control_type := ("0100", '0', '0', '0', '0', '0', '1', '1', '0', '0'); --0011
39 constant loadA: Control_type := ("0101", '0', '0', '0', '1', '0', '0', '0', '0', '0'); --0100
40 constant restore: Control_type := ("0110", '0', '0', '0', '0', '0', '0', '0', '0', '0'); --0101
41 constant save_restore: Control_type := ("0111", '0', '0', '0', '0', '0', '0', '1', '0', '0'); --0110
42 constant loadAahiftQCount: Control_type := ("0010", '1', '0', '0', '1', '0', '0', '0', '1', '0'); --0111
43 constant endstep: Control_type := ("1000", '0', '0', '0', '0', '0', '1', '0', '0', '1'); --1000
44
45 type Rom_type is array (0 to 8) of Control_type;
46 constant rom: Rom_type := (idle, init, shiftA, save_sub, loadA, restore, save_restore, loadAahiftQCount, endstep);
47 signal Control: Control_type;
48 begin
49     Control <= rom(to_integer(pc));
50     pc_next <= Control.pc_next;
51     shift_q <= Control.shift_q;
52     shift_a <= Control.shift_a;
53     load_q <= Control.load_q;
54     load_a <= Control.load_a;
55     start <= Control.start;
56     subtract <= Control.subtract;
57     save <= Control.save;
58     finish <= Control.finish;
59     count <= Control.count;
60
61
62 end synth;

```

Di seguito la descrizione di tutti gli stati:

- **idle**: tutti i segnali sono posti a 0, eccetto subtract che per è 1.
- **init**: avviene la load degli shift register e viene dato il segnale di start che serve per caricare nel registro A 0x0 invece che l'uscita dell'adder.
- **shiftA**: viene fatto lo shift a destra di A.
- **save_sub**: viene salvata l'uscita dell'adder in un registro temporaneo, in questo momento viene salvato anche il segno dell'uscita dell'adder in S.
- **loadA**: in A viene memorizzato il dato salvato nel registro temporaneo.
- **restore**: l'adder effettua l'operazione di somma invece che sottrazione.
- **save_restore**: viene salvato il valore dell'adder sul registro temporaneo a seguito di un restore.

- **loadAshiftQCount**: avviene lo shift di Q con ingresso $\overline{S}$, il salvataggio del registro temporaneo su A e viene incrementato il contatore.
- **endstep**: stato finale in cui viene messo a 1 il segnale di fine che causa l'assunzione da parte dell'uscita del risultato finale.

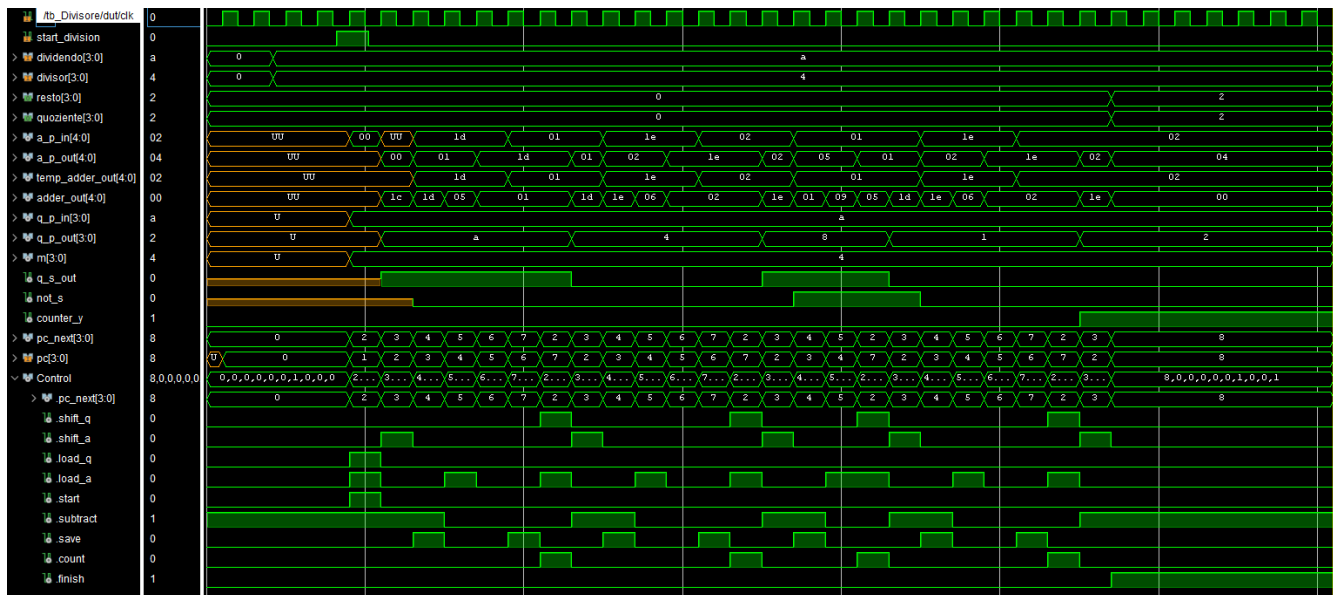
```

1 reg_pc: process(clk,reset)
2 begin
3   if rising_edge(clk) then
4     if reset = '1' then
5       pc <= "0000";
6     elsif restore = '0' and pc = "0100" then pc <= "0111";
7     elsif counter_y = '1' then pc <= "1000";
8     elsif start_division = '1' and (pc = "0000") then pc <= "0001";
9       dividendo_out <= dividendo_in; divisore_out <= divisore_in;
10    else pc <= pc_next;
11    end if;
12  end if;
13 end process;

```

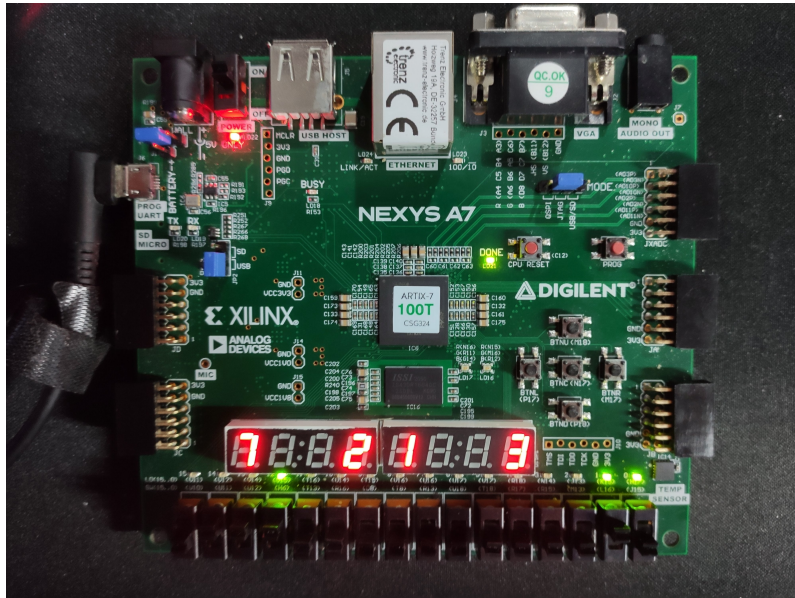
La control unit prevede anche alcuni salti, in particolare quando il segnale di restore è 0 e lo stato attuale è loadA allora si fa il jump all'operazione loadAshiftQCount. Allo stesso modo le assegnazioni iniziali degli operandi e l'uscita dallo stato di idle avvengono col segnale start_division. Infine si verifica l'uscita del contatore per portare il divisore nello stato finale.

10.3 Simulazione



Nella simulazione proviamo a dividere 0xA (10) per 0x2 (2), come previsto il risultato è 2 con resto di 2. Nella simulazione è possibile vedere i vari cambiamenti tra i vari stati definiti dal program counter della control unit e i segnali di controllo memorizzati sulla microROM.

10.4 Sintesi su board FPGA



Per quanto riguarda la sintesi sulla board non c'è molto d'aggiungere se non che gli ingressi vengono inseriti mediante gli switch e la divisione viene avviata tramite pulsante. I valori in ingresso vengono mostrati nella parte sinistra del display e i risultati su quella destra.

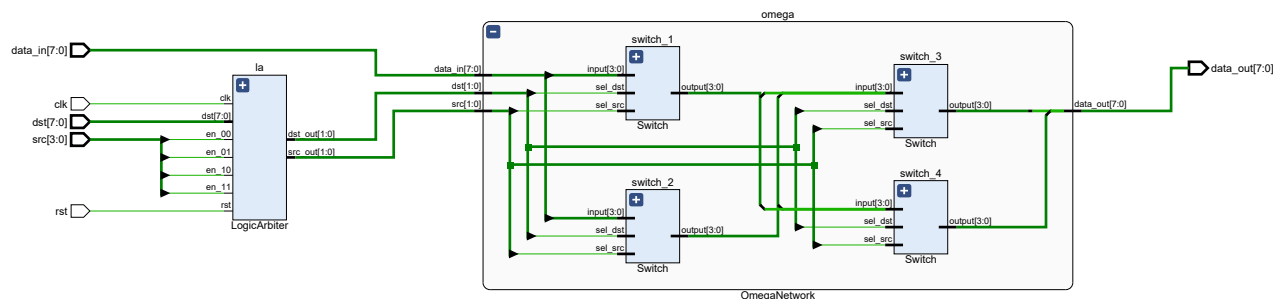
Capitolo 11

Switch Multistadio

Progettare ed implementare in VHDL uno switch multistadio secondo il modello omega network. Lo switch progettato deve operare come segue:

1. Lo switch deve consentire lo scambio di messaggi di 2 bit ciascuno da un nodo sorgente a un nodo destinazione in un rete con 4 nodi, implementando uno schema a priorità fissa fra i nodi (ed. nodo 1 più prioritario, con priorità decrescenti fino al nodo 4).
2. (Opzionale) rimuovendo l'ipotesi di lavorare secondo uno schema a priorità fra i nodi e considerando una rete di 8 nodi, lo switch deve gestire eventuali conflitti generati da collisioni secondo un meccanismo a scelta (ad es. perdendo uno dei messaggi in conflitto).
3. (Opzionale) Si implementi un protocollo di handshaking semplice regolato da una coppia di segnali (pronto a inviare/pronto a ricevere) per l'invio di ciascun messaggio fra due nodi.

11.1 Schematici



Lo switch è stato costruito a partire da due livelli di switch 2:2. Il primo livello deciderà l'indirizzamento dei dati in ingresso in base al primo bit dell'indirizzo della sorgente e della destinazione. Il secondo livello si baserà invece sul secondo bit quello meno significativo. Ogni switch è composto da un multiplexer 2:1 e da un demultiplexer 1:2 i cui segnali di selezione sono pilotati rispettivamente dalla bit di sorgente e dal bit di destinazione.

Il logic arbiter non fa altro che selezionare le abilitazioni in base a uno schema a priorità. Il dispositivo sorgente 00 è quello più prioritario.

11.2 Codice VHDL

```

1  entity LogicArbiter is
2      Port ( clk : in STD_LOGIC;
3            rst : in STD_LOGIC;
4            en_00: in std_logic;
5            en_01: in std_logic;
6            en_10: in std_logic;
7            en_11: in std_logic;
8            dst: in std_logic_vector(7 downto 0);
9            src_out: out std_logic_vector(1 downto 0);
10           dst_out: out std_logic_vector(1 downto 0)
11         );
12 end LogicArbiter;
13
14 architecture Dataflow of LogicArbiter is
15     signal temp_src, temp_dst: std_logic_vector(1 downto 0) := "00";
16 begin
17
18     temp_src <= "00" when en_00 = '1' else
19         "01" when en_01 = '1' else
20         "10" when en_10 = '1' else
21         "11" when en_11 = '1' else
22         "00";
23     temp_dst <= dst(1 downto 0) when en_00 = '1' else
24         dst(3 downto 2) when en_01 = '1' else
25         dst(5 downto 4) when en_10 = '1' else
26         dst(7 downto 6) when en_11 = '1' else
27         "00";
28
29     src_out <= temp_src when rising_edge(clk);
30     dst_out <= temp_dst when rising_edge(clk);
31 end Dataflow;

```

Per l'arbitro si è deciso di utilizzare un segnale di clock. Questa scelta è stata fatta arbitrariamente in quanto si può realizzare anche un circuito senza sincronismo.

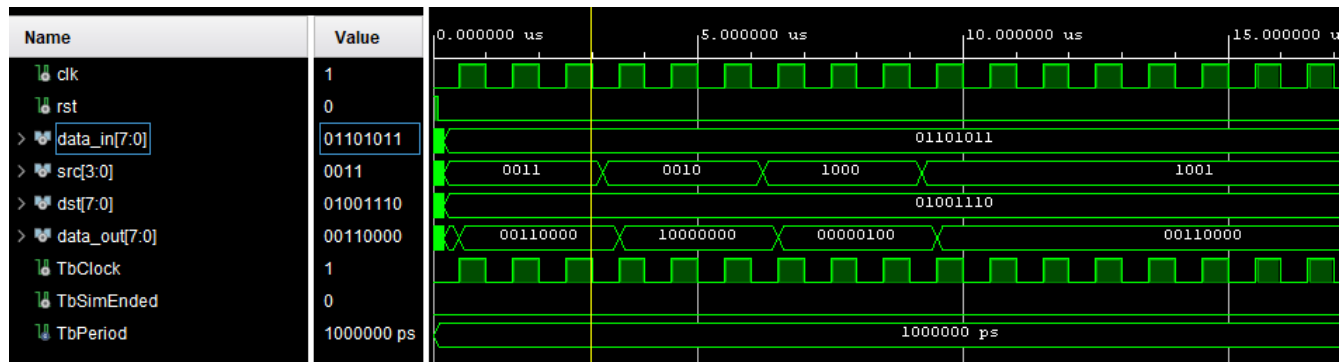
11.3 Simulazione

```

1  data_in <= "01101011";
2  src <= "0011";
3  dst <= "01001110";
4  wait for 3*TbPeriod;
5  src <= "0010";
6  wait for 3*TbPeriod;
7  src <= "1000";
8  wait for 3*TbPeriod;
9  src <= "1001";
10 wait for 10 * TbPeriod;

```

Per il testbench sono stati dati gli ingressi riportati sopra, è stata verificata sia la correttezza dell'instradamento dei dati sia il rispetto della politica prioritaria.



Capitolo 12

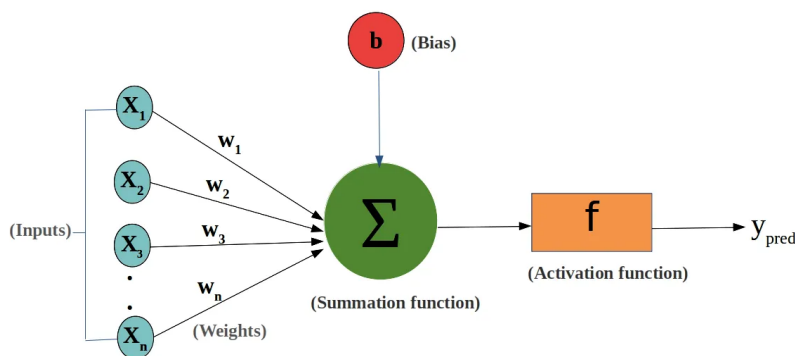
Rete Neurale (Esercizio Libero)

Viene richiesta l'implementazione di una rete neurale in grado di effettuare classificazioni sul dataset space shuttle. La rete neurale è un semplice multiplayer perceptron composto da un solo hidden layer.

Per implementare la rete neurale si è deciso di suddividere il problema partendo dall'unità elementare di una ANN ossia un singolo neurone (o perceptron).

12.1 Perceptron

Il perceptron è un'unità che dati un vettore di ingresso $x_0, x_1, \dots, x_n$ restituisce un'uscita y . Tale uscita è data dalla sommatoria degli ingressi moltiplicati per i relativi pesi $w_0, w_1, \dots, w_n$ sommata poi al bias b . Infine il risultato di queste operazioni viene usato come ingresso per una funzione di attivazione, in questo caso RELU.



La funzione RELU viene definita come.

$$f(x) = x^+ = \max(0, x) \quad (12.1)$$

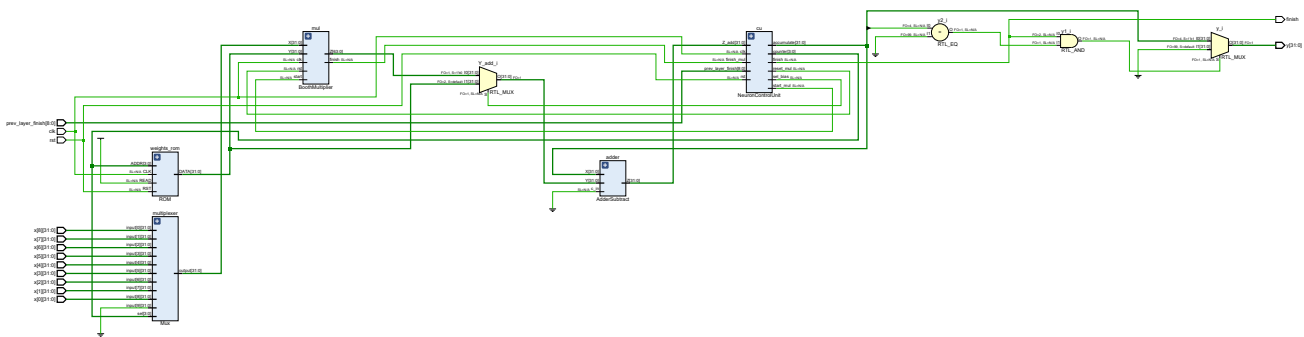
Ossia come il valore x se questo è positivo, altrimenti 0.

Gli elementi che compongono il neurone sono quindi un moltiplicatore, un addizionatore e la funzione RELU.

La prima scelta di progettazione effettuata è stata il livello di parallelismo interno. In linea di principio è possibile realizzare n moltiplicatori, uno per ingresso che effettuino il prodotto $x_i w_i$ e poi sommare tutte le uscite con addizionatori più o meno sofisticati. Si è scelto di non effettuare

questo tipo di scelta ma di realizzare un unico moltiplicatore che effettui uno dopo l'altro i prodotti tra ingressi e pesi e i cui risultati parziali vengano accumulati man mano in un registro. Quando poi le moltiplicazioni sono state tutte effettuate viene fatta la somma del bias e la funzione di attivazione. A causa di questa scelta c'è bisogno dell'introduzione di alcuni componenti aggiuntivi. Viene aggiunta una ROM che contenga i vari pesi e un'unità di controllo che coordini le operazioni di moltiplicazione e somma, inoltre viene aggiunto un contatore che tenga traccia del numero di moltiplicazioni effettuate e che allo stesso tempo scandisca gli accessi alla ROM. Chiaramente una scelta più spinta sul parallelismo del perceptron concettualmente ci avvicinerebbe di più a un ASIC dedicato solo ed esclusivamente all'esecuzione di questa rete neurale, mentre la scelta effettuata è concettualmente più vicina a una Neural Processing Unit.

12.1.1 Schematici



Nello schematico vediamo come sono collegati il moltiplicatore (di Booth), la control unit, l'adder e la ROM. Da segnalare la presenza di un paio di multiplexer e di porte logiche. Un multiplexer effettua la selezione tra il moltiplicatore e la ROM per effettuare l'operazione finale di addizione di bias. In uscita al perceptron un multiplexer seleziona come uscita il risultato del registro accumulatore o l'uscita nulla realizzando la funzione RELU. Tale selezione viene fatta in base all'ultimo bit del registro accumulatore, quando questo è alto il risultato è negativo e quindi in uscita viene restituito 0.

Tra i vari segnali in ingresso alla CU vi è anche un vettore di segnali che verificano che i neuroni del layer precedente abbiano completato le operazioni. Quando tutti i bit sono alti vengono iniziate le operazioni del neurone. A sua volta il neurone metterà alto il suo bit di finish quando i calcoli saranno completati.

12.1.2 Codice

```
1 package MyPackage is
2   subtype neuron_input is std_logic_vector(31 downto 0);
3   type neuron_input_vector is array (integer range <>) of neuron_input;
4 end MyPackage;
```

Per facilitare la leggibilità del codice sono stati definiti un paio di tipi personalizzati, ossia `neuron_input` e `neuron_input_vector`, il primo è un alias di `std_logic_vector(31 downto 0)` mentre il secondo è un vettore del primo.

```

1  entity NeuronControlUnit is generic(
2      prev_layer_size: integer := 9
3  );
4      Port ( clk : in STD_LOGIC;
5             rst : in STD_LOGIC;
6             sel_bias: out std_logic;
7             finish : out STD_LOGIC;
8             start_mul: out std_logic;
9             reset_mul: out std_logic;
10            Z_add: in neuron_input;
11            accumulate: out neuron_input;
12            finish_mul : in std_logic;
13            prev_layer_finish : in STD_LOGIC_VECTOR (prev_layer_size-1 downto 0);
14            counter: out std_logic_vector(3 downto 0)
15        );
16
17 end NeuronControlUnit;
18
19 architecture Behavioral of NeuronControlUnit is
20
21     Type State is (IDLE, RST_MULT, MULT, WAIT_MULT, ACC, BIAS, ACC_BIAS ,FIN);
22     signal current_state, next_state : State := IDLE;
23
24 begin
25     mem: process(clk, rst)
26     begin
27         if rst = '1' then
28             current_state <= IDLE;
29         elsif rising_edge(clk) then
30             current_state <= next_state;
31         end if;
32     end process;
33
34     fsm: process(current_state, finish_mul, prev_layer_finish)
35     variable all_prev_finish: std_logic := '0';
36     variable count: integer := 0;
37     begin
38
39         case current_state is
40             when IDLE =>
41                 finish <= '0';
42                 all_prev_finish := prev_layer_finish(0) and prev_layer_finish(1);
43                 for i in 2 to prev_layer_size-1 loop
44                     all_prev_finish := all_prev_finish and prev_layer_finish(i);
45                 end loop;
46                 if all_prev_finish = '1' then
47                     accumulate <= (others => '0');
48                     next_state <= RST_MULT;
49                 end if;
50             when RST_MULT =>
51                 sel_bias <= '0';
52                 reset_mul <= '1';
53                 next_state <= MULT;
54             when MULT =>
55                 reset_mul <= '0';
56                 start_mul <= '1';
57                 next_state <= WAIT_MULT;
58             when WAIT_MULT =>
59                 start_mul <= '0';
60                 if finish_mul = '1' then
61                     next_state <= ACC;
62                 end if;
63             when ACC =>
64                 accumulate <= Z_add;
65                 count := count+1;
66                 if count = (prev_layer_size) then
67                     next_state <= BIAS;
68                 else
69                     next_state <= RST_MULT;
70                 end if;
71             when BIAS =>
72                 sel_bias <= '1';
73                 next_state <= ACC_BIAS;
74             when ACC_BIAS =>
75                 accumulate <= Z_add;
76                 next_state <= FIN;
77             when FIN =>
78                 finish <= '1';
79         end case;
80         counter <= std_logic_vector(to_unsigned(count, 4));
81     end process;
82
83 end Behavioral;
84 ;

```

L'automa a stati finiti invece è molto semplice. La CU inizialmente si trova in IDLE, quando tutti i segnali di finish del layer precedente sono pari alti transita allo stato RST_MULT in cui viene fatto il reset del moltiplicatore, dopodichè si passa allo stato MULT in cui viene dato il segnale di start al moltiplicatore. WAIT_MULT è lo stato in cui si passa più tempo e si attende che venga

fatta la moltiplicazione tra due valori a 32 bit, se arriva il segnale di `finish` dal moltiplicatore si passa allo stato ACC dove viene memorizzato il risultato parziale in uscita dall'adder.

Un contatore (comportamentale) tiene traccia del numero di moltiplicazioni effettuate ed è usato anche come indirizzo della ROM, quando tutte le moltiplicazioni sono state fatte si passa allo stato BIAS per cambiare l'ingresso dell'adder dal moltiplicatore alla memoria (che in uscita ha il bias). Lo stato ACC_BIAS memorizza il risultato dell'addizionatore e infine si passa allo stato FIN dove si segnala la fine delle operazioni.

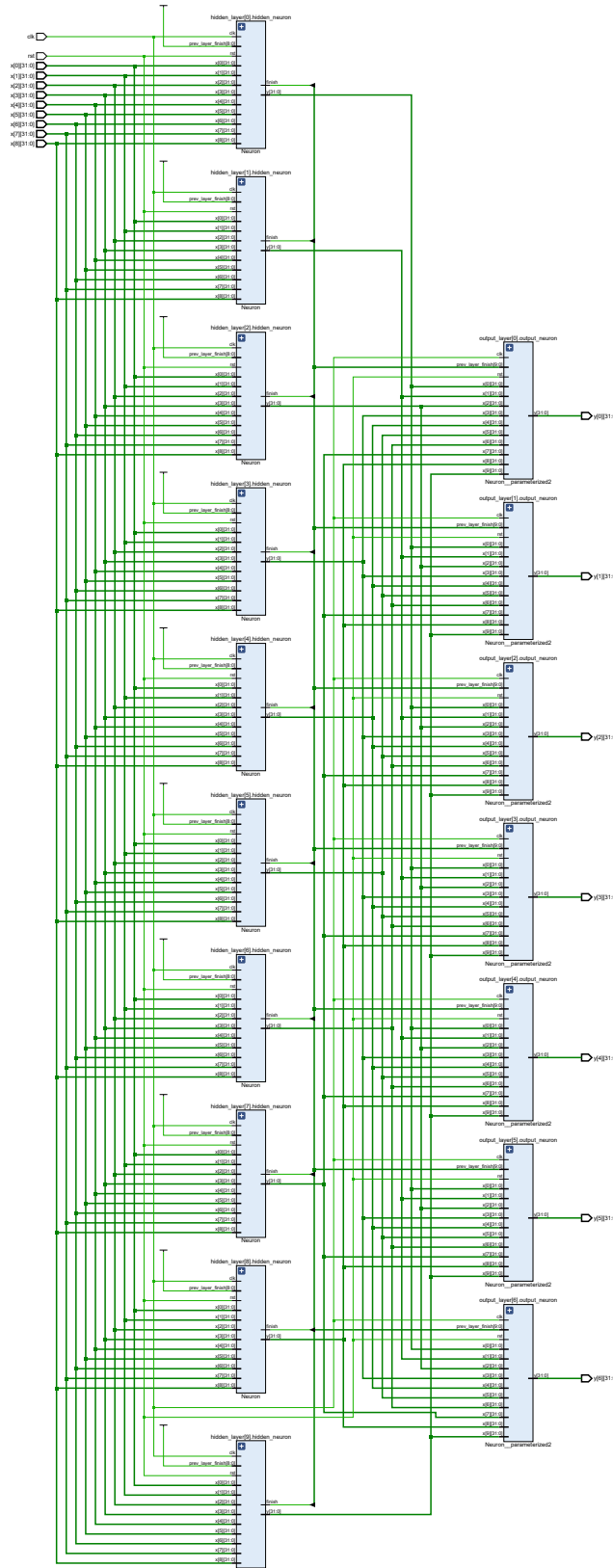
12.1.3 Simulazione



Nella simulazione possiamo vedere come la maggior parte del tempo venga impiegata per effettuare i vari prodotti tra pesi e ingressi. Nella terza e seconda immagine vengono mostrati il dettaglio delle transizioni tra i vari stati.

12.2 Rete neurale

12.2.1 Schematici



Combinando più perceptron insieme viene ricostruito quello che è il layout del classificatore. I pesi e i bias per ogni neurone vengono caricati da file sulle ROM di ciascuno di essi.

12.2.2 Codice

```

1  entity NeuralNetwork is generic (n_features: integer := 9; hidden_layer_size: integer := 10; n_classes: integer
   := 7);
2      Port ( clk : in STD_LOGIC;
3            rst : in STD_LOGIC;
4            x: in neuron_input_vector(n_features-1 downto 0);
5            y: out neuron_input_vector(n_classes-1 downto 0)
6            );
7  end NeuralNetwork;
8
9  architecture Structural of NeuralNetwork is
10 component Neuron
11     generic(
12         prev_layer_size: integer := 9;
13         weights_path: string := ""
14     );
15     Port ( clk : in STD_LOGIC;
16           rst : in STD_LOGIC;
17           x: in neuron_input_vector(prev_layer_size-1 downto 0);
18           prev_layer_finish : in std_logic_vector(prev_layer_size-1 downto 0);
19           y: out neuron_input;
20           finish: out std_logic
21         );
22 end component;
23
24 signal hidden_layer_out: neuron_input_vector(hidden_layer_size-1 downto 0);
25 signal hidden_layer_finish: std_logic_vector (hidden_layer_size-1 downto 0);
26 begin
27
28 hidden_layer: for i in 0 to hidden_layer_size-1 generate
29     hidden_neuron: Neuron generic map(prev_layer_size => n_features, weights_path => "hidden_layer-" & integer'
30                                     image(i) & ".txt")
31     port map(
32         clk => clk,
33         rst => rst,
34         x => x,
35         y => hidden_layer_out(i),
36         prev_layer_finish => (others => '1'),
37         finish => hidden_layer_finish(i)
38     );
39 end generate;
40
41 output_layer: for i in 0 to n_classes-1 generate
42     output_neuron: Neuron generic map(prev_layer_size => hidden_layer_size, weights_path => "output_layer-" &
43                                     integer'image(i) & ".txt")
44     port map(
45         clk => clk,
46         rst => rst,
47         x => hidden_layer_out,
48         y => y(i),
49         prev_layer_finish => hidden_layer_finish,
50         finish => open
51     );
52 end generate;
53 end Structural;

```

I vari neuroni vengono creati con un costrutto for-generate usando dei generic che ricevono il numero di neuroni del layer nascosto, il numero di classi e di features.

12.2.3 Simulazione

Nella simulazione vediamo l'esecuzione con in ingresso un campione del dataset. Ogni uscita rappresenta la probabilità che quel campione appartenga a una determinata feature (in questa ANN manca un layer come ad esempio softmax, quindi le probabilità non sono state normalizzate).